Оглавление

1. Подготовка и загрузка данных 2
Excel 2
SPSS 3
SQL 4
Python 5
A. Описание и классификация переменных 6
Excel 8
SPSS 12
SQL 16
Python 19
B. Формирование выборок 24
C. Количественная репрезентативность 26
D. Доверительные интервалы 27
E. ЦПТ и Закон больших чисел 29
F. Параметрические оценки 33
G. Свойства оценок и обсуждение 36
H. Бизнес-вывод 39
Оценка наличия сезонности 40
Оценка эффективности каждой рекламной кампании 42
Оценка эффективности каждого трафика 43
Гендерные различия 44
Оценка зависимости суммы покупки от времени и глубины посещения сайта 45
NPS по месяцам 46
Выводы и рекомендации 46

Проект по статистике

Толстяков Роман

Каждую из задач проекта будем решать с помощью нескольких инструментов:

- Exce
- SPSS
- SQL
- Python

Дублирование в данном случае делается осознанно, чтобы сравнить инструменты и для будущей аналитической работы в зависимости от масштаба и сложности задачи выбирать оптимальный стек инструментов. А также дополнительно сравнить результаты решения и оценить возможное наличие расхождений и удобство визуализации.

1. Подготовка и загрузка данных

Загрузите датасет и опишите каждый столбец (тип данных)

Excel

Так как разделителем в исходном датасет является ; , то файл загружается как есть без необходимости в дальнейшем разделять ячейки на столбцы (Данные / Текст по столбцам).

B4 | 01.01.2025

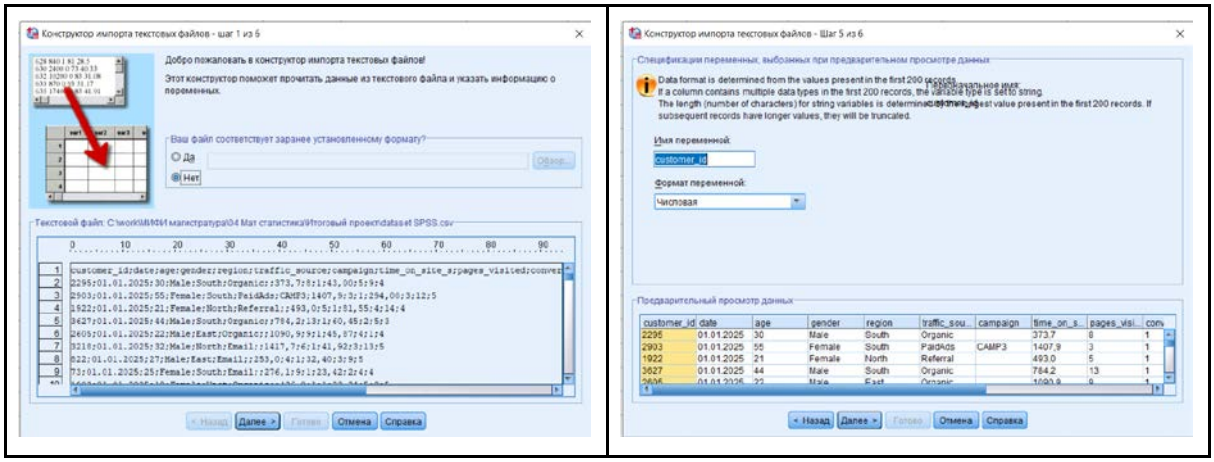
	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	customer_	date	age	gender	region	traffic_sou	campaign	time_on_s	pages_visi	converted	purchase_	num_item	delivery_ti	satisfaction
2	2295	01.01.2025	30	Male	South	Organic		373,7	8	1	43	5	9	4
3	2903	01.01.2025	55	Female	South	PaidAds	CAMP3	1407,9	3	1	294	3	12	5
4	1922	01.01.2025	21	Female	North	Referral		493	5	1	81,55	4	14	4
5	3627	01.01.2025	44	Male	South	Organic		784,2	13	1	60,45	2	5	3
6	2605	01.01.2025	22	Male	East	Organic		1090,9	9	1	45,87	4	1	4
7	3218	01.01.2025	32	Male	North	Email		1417,7	6	1	41,92	3	13	5
8	822	01.01.2025	27	Male	East	Email		253	4	1	32,4	3	9	5
9	73	01.01.2025	25	Female	South	Email		276,1	9	1	23,42	2	4	4
10	1902	01.01.2025	18	Female	West	Organic		436	1	1	23,34	5	8	5
11	4979	01.01.2025	67	Male	East	Organic		198,9	4	1	20,94	7	9	5
12	1427	01.01.2025	23	Male	North	Organic		80,3	6	1	20,31	4	8	5
13	2633	01.01.2025	36	Female	South	PaidAds	CAMP5	483,7	3	0	0	0	0	
14	4777	01.01.2025	55	Male	South	Organic		466,4	7	0	0	0	0	
15	4412	01.01.2025	58	Male	South	PaidAds	CAMP3	454	3	0	0	0	0	
16	2328	01.01.2025	39	Female	East	Organic		180,4	11	0	0	0	0	
17	4152	01.01.2025	45	Male	North	Organic		168,2	5	0	0	0	0	
18	4130	01.01.2025	57	Male	West	PaidAds	CAMP2	131	7	0	0	0	0	
19	2612	01.01.2025	29	Other	East	Email		128,2	4	0	0	0	0	
20	2912	01.01.2025	21	Female	East	Organic		114,4	2	0	0	0	0	

Остальные типы данных определены автоматически (это связано с тем, что в исходном файле разделителем в дробной части была запятая)

customer_id	ID пользователя.	целое число
Date	дата посещения сайта пользователем.	дата (целое число)
age	возраст пользователя.	целое число
gender	пол	строка
region	регион проживания пользователя	строка
traffic_source	источник трафика	строка
campaign	рекламная кампания	строка
time_on_site_s	сколько секунд клиент провёл на сайте.	число (с точкой)
pages_visited	число просмотренных страниц за визит.	целое число
converted	совершил клиент покупку или нет.	бинарное
purchase_amoun	сумма покупки	число (с точкой)
num_items	сколько товаров куплено в заказе	целое число
delivery_time_da	время доставки заказа (дни).	целое число
satisfaction	оценка удовлетворённости клиента.	целое число

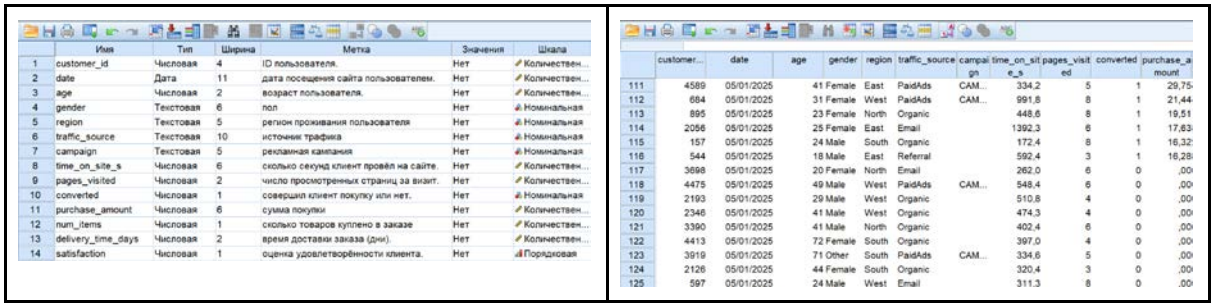
SPSS

Загрузка исходного файла может быть осуществлена либо через конструктор импорта текстовых файлов либо методом переноса через буфер обмена листа данных из Excel. Первый способ является более универсальным, так как на стадии импорта позволяет определить типы переменных.



Но текстовые переменные (пол, регион, источник трафика) вставляются именно как текст. Для перевода их в числовые требуется дополнительная обработка это сократит время обработки, но в конкретно данном случае не является критичным

gender
Male - 1
Female - 2



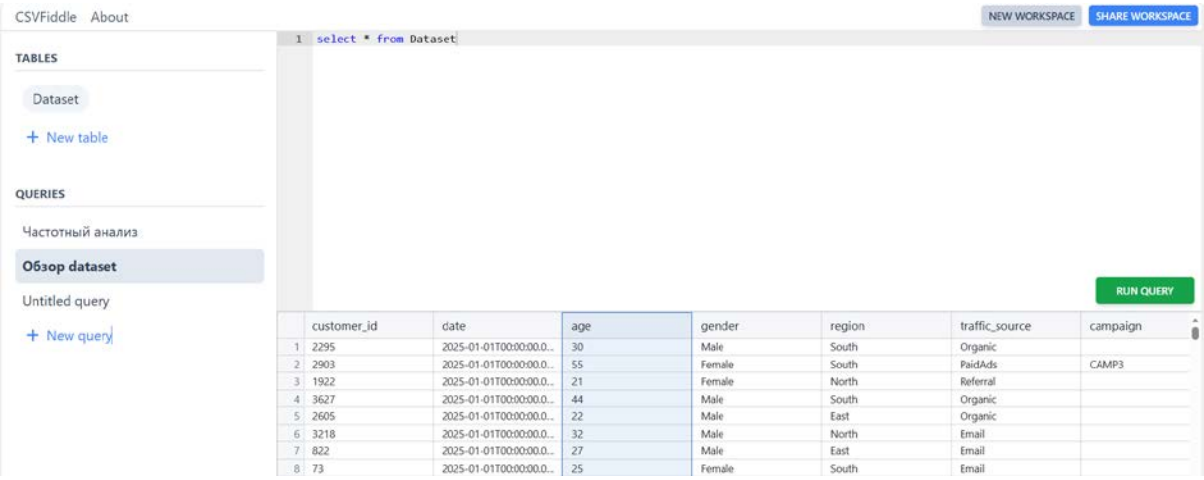
	Имя	Тип	Ширина	Метка	Значения	Шкала
1	customer_id	Числовая	4	ID пользователя.	Нет	Количествен...
2	date	Дата	11	дата посещения сайта пользователем.	Нет	Количествен...
3	age	Числовая	2	возраст пользователя.	Нет	Количествен...
4	gender	Текстовая	6	пол	Нет	Номинальная
5	region	Текстовая	5	регион проживания пользователя	Нет	Номинальная
6	traffic_source	Текстовая	10	источник трафика	Нет	Номинальная
7	campaign	Текстовая	5	рекламная кампания	Нет	Номинальная
8	time_on_site_s	Числовая	6	сколько секунд клиент провёл на сайте.	Нет	Количествен...
9	pages_visited	Числовая	2	число просмотренных страниц за визит.	Нет	Количествен...
10	converted	Числовая	1	совершил клиент покупку или нет.	Нет	Номинальная
11	purchase_amount	Числовая	6	сумма покупки	Нет	Количествен...
12	num_items	Числовая	1	сколько товаров куплено в заказе	Нет	Количествен...
13	delivery_time_days	Числовая	2	время доставки заказа (дни).	Нет	Количествен...
14	satisfaction	Числовая	1	оценка удовлетворённости клиента.	Нет	Порядковая

	customer...	date	age	gender	region	traffic_source	campaign	time_on_site_s	pages_visited	converted	purchase_amount
111	4589	05/01/2025	41	Female	East	PaidAds	CAM...	334,2	5	1	29,75
112	684	05/01/2025	31	Female	West	PaidAds	CAM...	991,8	8	1	21,44
113	895	05/01/2025	23	Female	North	Organic		448,6	8	1	19,51
114	2056	05/01/2025	25	Female	East	Email		1392,3	6	1	17,63
115	157	05/01/2025	24	Male	South	Organic		172,4	8	1	16,32
116	544	05/01/2025	18	Male	East	Referral		592,4	3	1	16,28
117	3698	05/01/2025	20	Female	North	Email		262,0	6	0	,00
118	4475	05/01/2025	49	Male	West	PaidAds	CAM...	548,4	6	0	,00
119	2193	05/01/2025	29	Male	West	Organic		510,8	4	0	,00
120	2346	05/01/2025	41	Male	West	Organic		474,3	4	0	,00
121	3390	05/01/2025	41	Male	North	Organic		402,4	6	0	,00
122	4413	05/01/2025	72	Female	South	Organic		397,0	4	0	,00
123	3919	05/01/2025	71	Other	South	PaidAds	CAM...	334,6	5	0	,00
124	2126	05/01/2025	44	Female	South	Organic		320,4	3	0	,00
125	597	05/01/2025	24	Male	West	Email		311,3	8	0	,00

По сравнению с Excel загрузка в SPSS несколько сложнее

SQL

Для обработки данных через SQL воспользуемся сервисом https://csvfiddle.io/ - для csv файлов - это максимально удобный и быстрый инструмент. Но требует подключения к интернет. Для автономной работы потребовалось бы создание таблицы в БД, развернутой на localhost



Для проверки какой тип переменных у каждого столбца (есть риск, что при конвертации часть данных импортировались как строка) выполним запрос.

```
select  pg_typeof(customer_id)  as customer_id ,
        pg_typeof("date")  as "date" ,
        pg_typeof(age)  as age ,
        pg_typeof(gender)  as gender ,
        pg_typeof(region)  as region ,
        pg_typeof(traffic_source)  as traffic_source ,
        pg_typeof(campaign)  as campaign ,
        pg_typeof(time_on_site_s)  as time_on_site_s ,
        pg_typeof(pages_visited)  as pages_visited ,
        pg_typeof(converted)  as converted ,
        pg_typeof(purchase_amount)  as purchase_amount ,
        pg_typeof(num_items)  as num_items ,
        pg_typeof(delivery_time_days)  as delivery_time_days ,
        pg_typeof(satisfaction)  as satisfaction
from Dataset
limit 1
```

	customer_id	date	age	gen...	region	traffic...	camp...	time_o...	pages_...	conve...	purch...	num...	deliv...	satisfa...
1	integer	date	integer	varchar	varchar	varchar	varchar	varchar	integer	integer	varchar	integer	integer	integer

Поля time_on_site_s и purchase_amount (наличие запятой в качестве разделителя целой и дробной части) импортированы как строка, при дальнейшем анализе их необходимо привести к типу numeric. Но так как https://csvfiddle.io/ работает с csv файлом как с таблицей и создание новой таблицы возможно лишь виртуально, то проще заменить “,” на “.” в базовом файле и загрузить по-новой.

Повторная проверка показала, что данные загружены корректно, типы данных определены правильно.

RUN QUERY

	customer_id	date	age	gender	region	traffic_source	campaign	time_on_site...	pages_visit...	converted	purchase_amount	num_items	delivery_time_da...	satisfaction
1	integer	date	integer	varchar	varchar	varchar	varchar	double	integer	integer	double	integer	integer	integer

Python

Будем использовать Google Collab (это также не автономное решение, но нас так учили и мне нравится □). Чтобы исключить “проблем” с типами файла зададим типы при загрузке данных явным образом
ссылка на блокнот

```
import pandas as pd

def comma_to_float(x):
    """Конвертирует строку с запятой в float"""
    try:
        if pd.isna(x) or str(x).strip() == '':
            return None
        # Убираем пробелы и заменяем запятую на точку
        return float(str(x).replace(',', '.'))
    except:
        return None

ecom = pd.read_csv(                 #Парсим данные, сразу в правильные типы
    'dataset.csv',
    sep=';',
    encoding='utf-8',
    parse_dates=['date'],           #Дата парсится отдельно
    dayfirst=True,
    dtype={                         #строки и целые типы
        'customer_id': 'Int64',     #Int64 (с возможностью пропуска данных)
        'age': 'Int64',
        'gender': 'str',
        'region': 'str',
        'traffic_source': 'str',
        'campaign': 'str',
        'pages_visited':'Int64',
```

```
        'converted':'Int64',
        'num_items': 'Int64',
        'delivery_time_days': 'Int64',
        'satisfaction': 'Int64'
        },
    converters={                    #десятичные числа, сначала конвертируем
        'purchase_amount': comma_to_float,
        'time_on_site_s': comma_to_float
    }
)
```

Если не учитывать тот факт, что Python для меня пока новый инструмент и для написания этих двух строк пришлось долго вспоминать и гуглить особенно “фишки” с датой и запятыми в точки, то это все равно это довольно простое решение.

А. Описание и классификация переменных

Классифицируйте переменные: количественные/категориальные/порядковые/бинарные.

Вне зависимости от инструмента анализа классификация везде будет идентична (в SPSS она уже сделана)

customer_id - номинальная переменная, из предлагаемого списка относится скорее к категориальной, так как не несет количественной нагрузки, а просто фиксирует номер пользователя;

date - количественная переменная, так как с увеличением или уменьшением ее величины меняется сущность значения - раньше или позже посещался сайт, хотя дата может использоваться для формирования кагорт и тогда она выступает как категориальная переменная;

age - по аналогии с датой, также количественная переменная;

gender - номинальная категориальная переменная, так как разделяет выборку на две категории по полу;

region, traffic_source, campaign - по аналогии с gender также являются номинальными категориальными переменными;

time_on_site_s, pages_visited, purchase_amount, num_items, delivery_time_days - количественные переменные, так как содержат количественную оценку того или иного действия пользователя Для ключевых переменных (purchase_amount, time_on_site, pages_visited, delivery_time, satisfaction) постройте гистограммы и посчитайте среднее, медиану, дисперсию, коэффициент вариации.

converted - бинарная (дихотомическая) переменная, выступает в роли флага совершения покупки. Интерес представляют только значения = 1

satisfaction - порядковая переменная, так как удовлетворенность понятие относительное, не имеющее количественных единиц измерений, поэтому ее

оценивают в баллах. 1 -низкая, 2 - выше и т.д. с ростом значения переменной фиксируется более высока удовлетворенность

Для ключевых переменных (purchase_amount, time_on_site, pages_visited, delivery_time, satisfaction) постройте гистограммы и посчитайте среднее, медиану, дисперсию, коэффициент вариации.

Excel

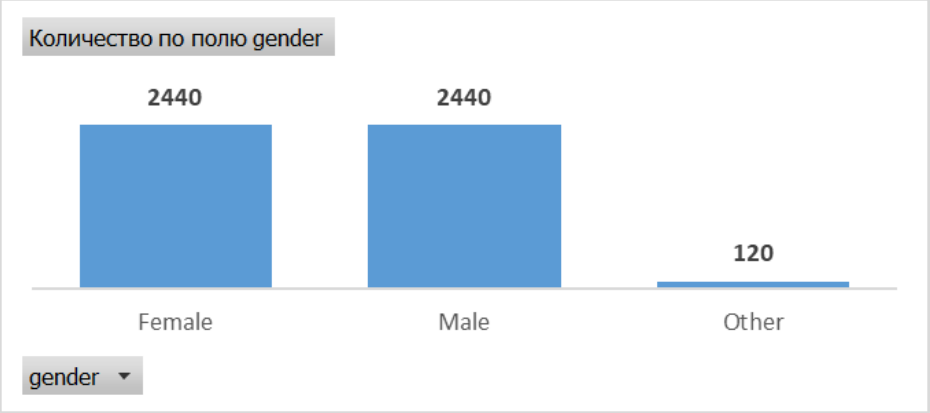
Частотный анализ для категориальных и порядковых значений проще всего проводить при помощи сводных таблиц.

Дата

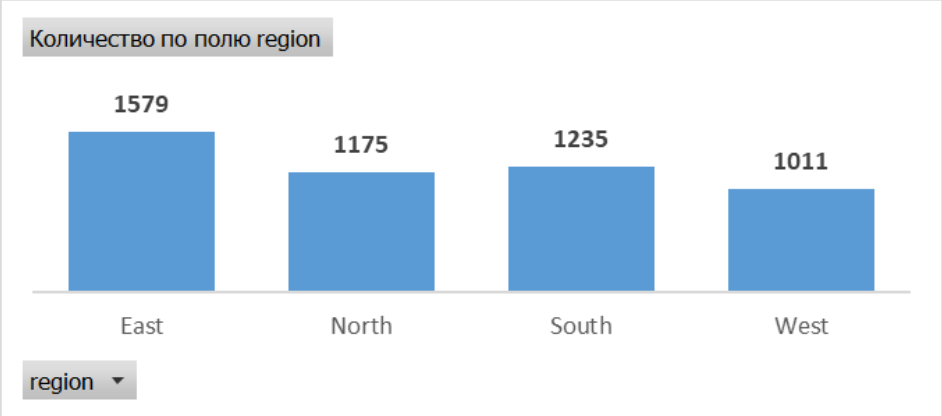
Дата	Заходы
01.01.2025	23
02.01.2025	29
03.01.2025	31
04.01.2025	26
05.01.2025	33
06.01.2025	24
07.01.2025	22
08.01.2025	38
09.01.2025	23
10.01.2025	30
11.01.2025	32
12.01.2025	34
13.01.2025	41
14.01.2025	31
15.01.2025	32
16.01.2025	32
17.01.2025	24

Период	01.01.2025	29.06.2025
Среднее количество заходов на сайт за день	27,8	

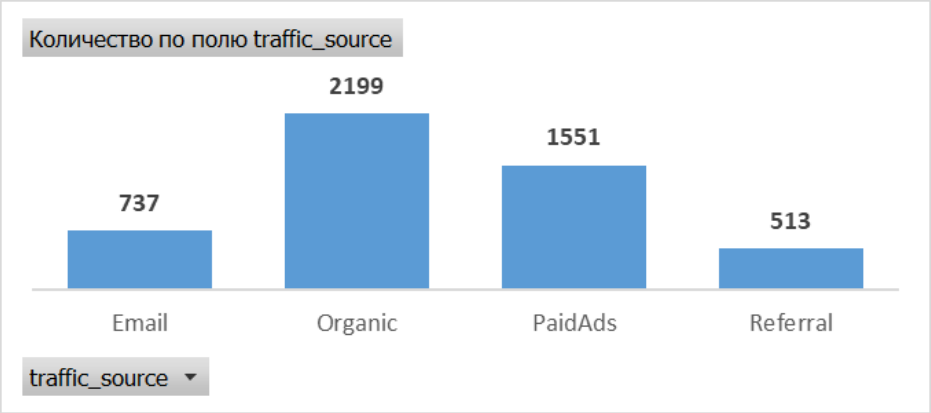
Пол



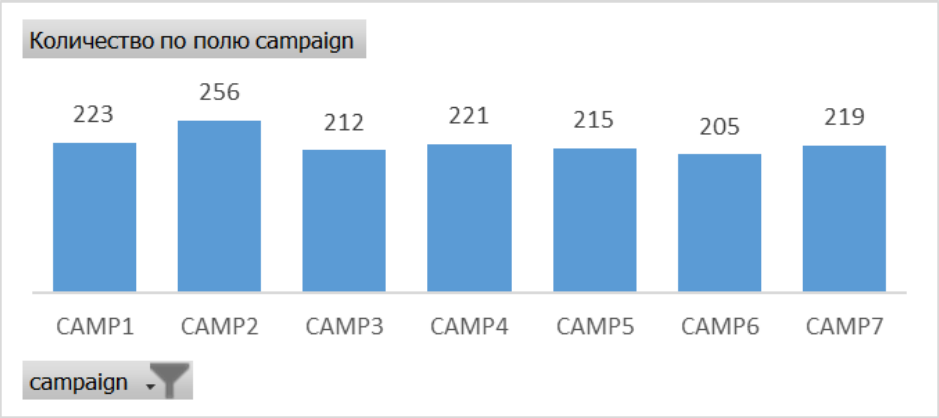
Регион



Источник трафика



Рекламная кампания




```
Для количественных используются стандартные формулы:
=МИН(datasetExcel!C:C)
=МАКС(datasetExcel!C:C)
=СРЗНАЧ(datasetExcel!C:C)
=МЕДИАНА(datasetExcel!C:C)
=ДИСПР(datasetExcel!C:C)
=СТАНДОТКЛОН(datasetExcel!C:C)/СРЗНАЧ(datasetExcel!C:C)
```

Характериситка	purchase_amount	time_on_site_s	pages_visited	delivery_time_days	satisfaction
Диапазон от:	0	1	1	0	2
Диапазон до:	1419,3	2235,3	18	15	5
Среднее	34,9278	361,8725	5,9884	2,8718	4,0951
Медиана	0,0000	305,4500	6,0000	0,0000	4,0000
Дисперсия	7613,5259	65269,4028	5,0627	20,9242	0,4146
Коэффицент вариации	2,4984	0,7061	0,3758	1,5930	0,1573

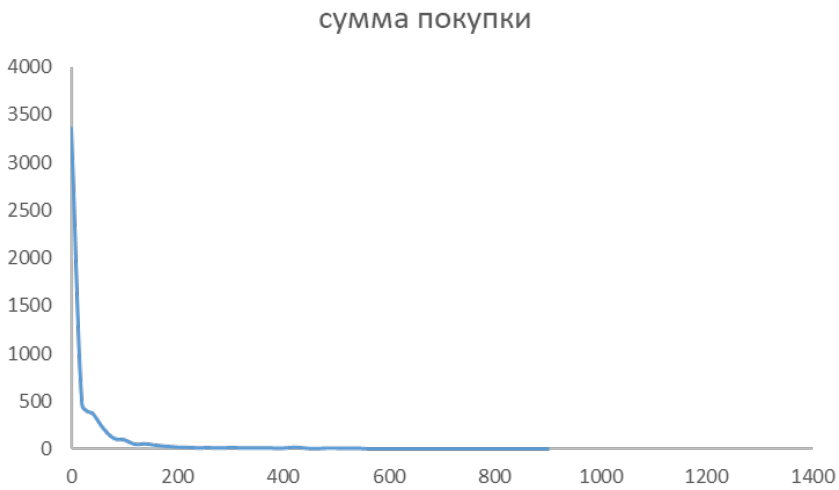
В данном примере статистика рассчитана по всем пользователям, включая тех кто не совершал покупок и не посматривал страниц, а таких было достаточно много, о чем свидетельствует 0 значение медианы по полям

- сумма покупки
- время доставки заказа (дни).

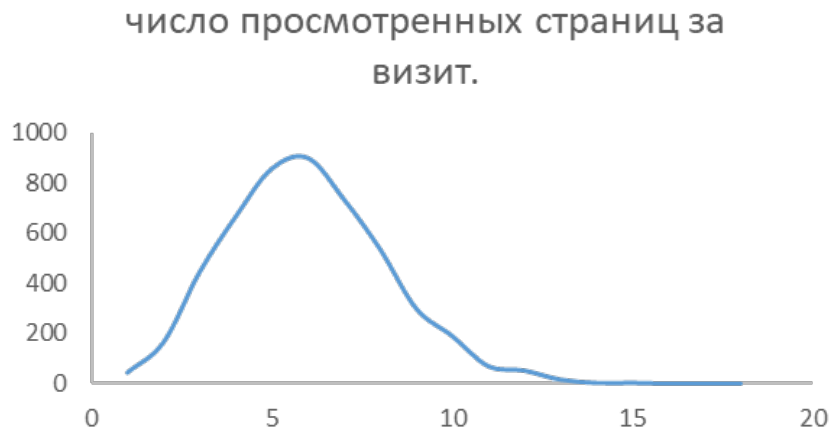
В данном случае такие данные нельзя назвать корректными и статистику по пользователям и покупателям необходимо считать на разных уровнях воронки. Несмотря на то что в Excel (2019 версия) имеется тип графика “Гистограмма” с автоматической агрегацией данных по диапазонам, более наглядную картину дают графики, построенные с агрегацией данных в сводных таблицах. Это связано со спецификой выбора групп внутри гистограммы для агрегации и невозможностью управлять этим процессом.

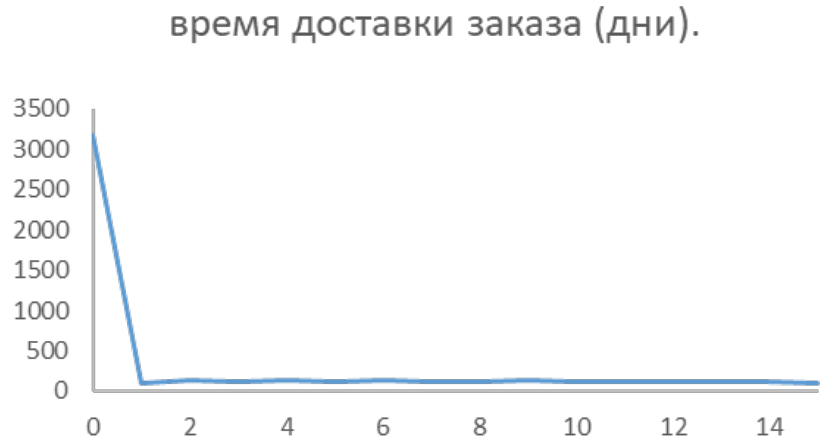
стандартная Гистограмма по столбцу Pages_visited

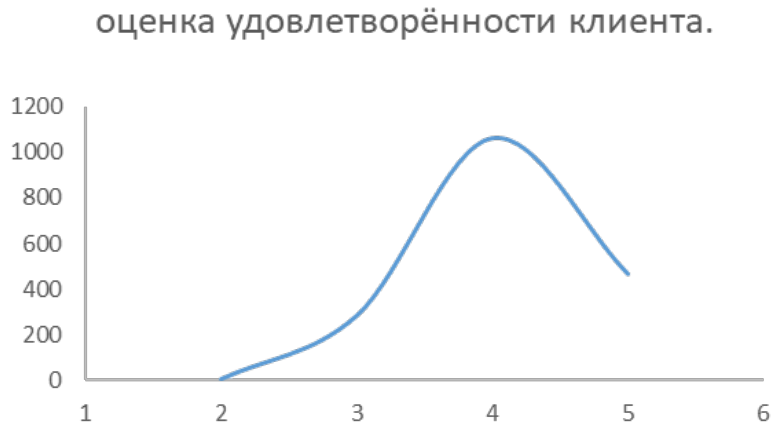
Поэтому для визуализации воспользуемся данными из сводных таблиц, но оформим это как точечный график для соблюдения пропорций по оси абсцис (нулевые значения при этом игнорируются)



сколько секунд клиент провёл на сайте.
700
600
500
400
300
200
100
0
-100
0
500
1000
1500
2000
2500


число просмотренных страниц за визит.
1000
800
600
400
200
0
0
5
10
15
20


время доставки заказа (дни).
3500
3000
2500
2000
1500
1000
500
0
0
2
4
6
8
10
12
14



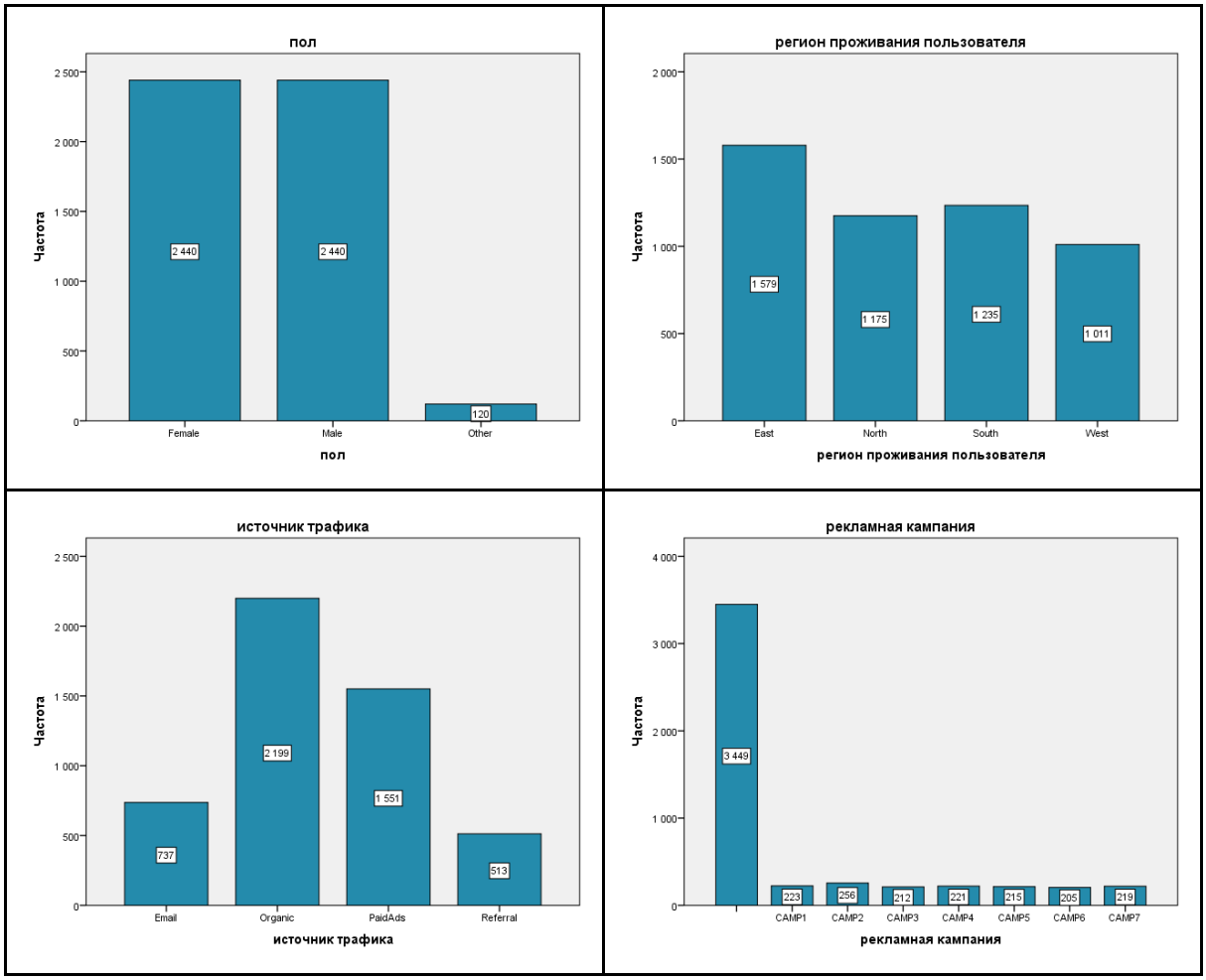
Несмотря на очевидность и простоту необходимых действий, процесс построения сводных таблиц, переноса данных и визуализации данных занял избыточно много времени.

SPSS

Частотный анализ для переменных любых типов реализуется через меню
Анализ / Описательный анализ / Частоты

Но для быстрой визуализации таблиц без дополнительной статистики (для номинальных и порядковых переменных) целесообразно использовать конструктор таблиц.

		Частоты	Итоговый % по таблице
пол	Female	2440	48,8%
	Male	2440	48,8%
	Other	120	2,4%
регион проживания пользователя	East	1579	31,6%
	North	1175	23,5%
	South	1235	24,7%
	West	1011	20,2%
источник трафика	Email	737	14,7%
	Organic	2199	44,0%
	PaidAds	1551	31,0%
	Referral	513	10,3%
рекламная кампания		3449	69,0%
	CAMP1	223	4,5%
	CAMP2	256	5,1%
	CAMP3	212	4,2%
	CAMP4	221	4,4%
	CAMP5	215	4,3%
	CAMP6	205	4,1%
	CAMP7	219	4,4%

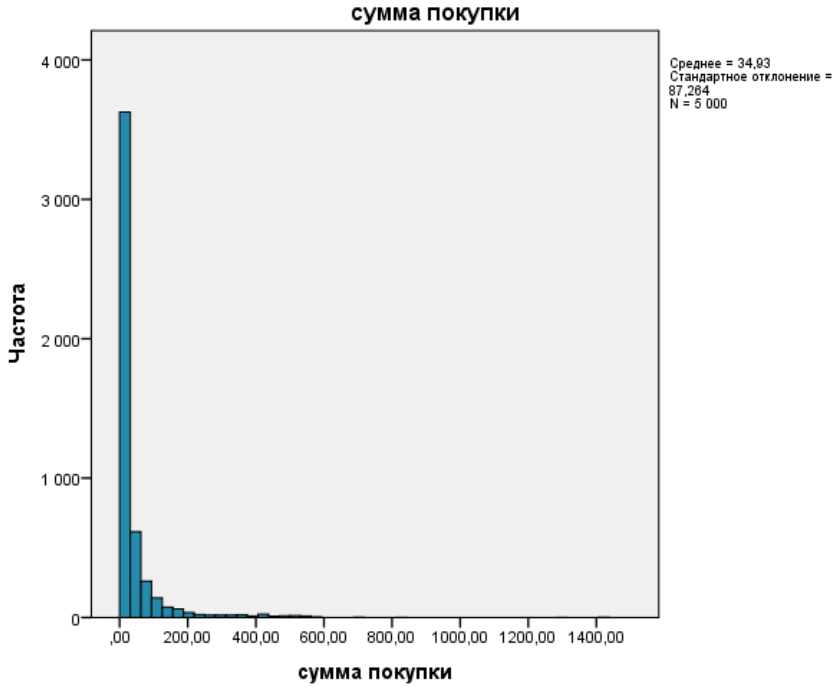


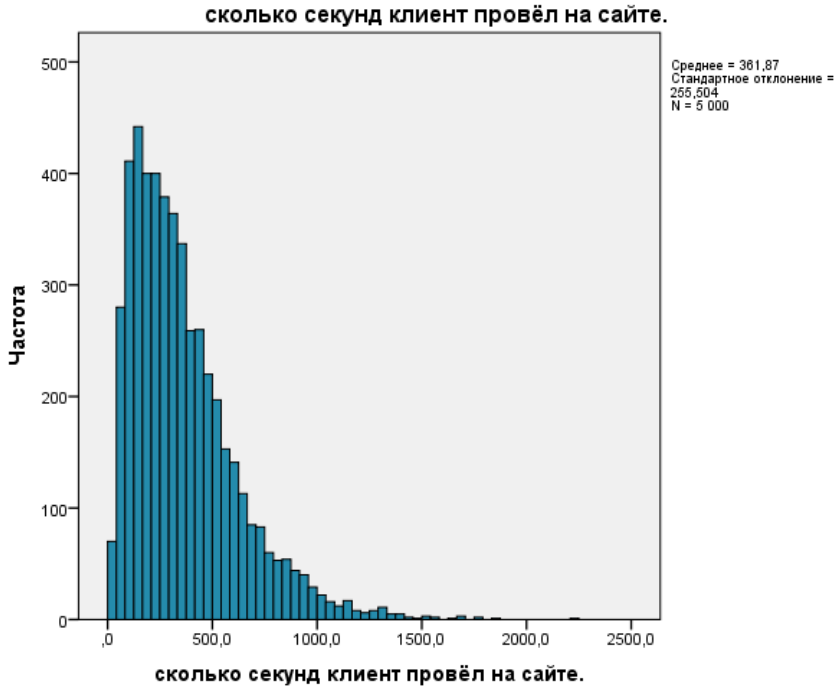
Расчет статистики для количественных переменных:

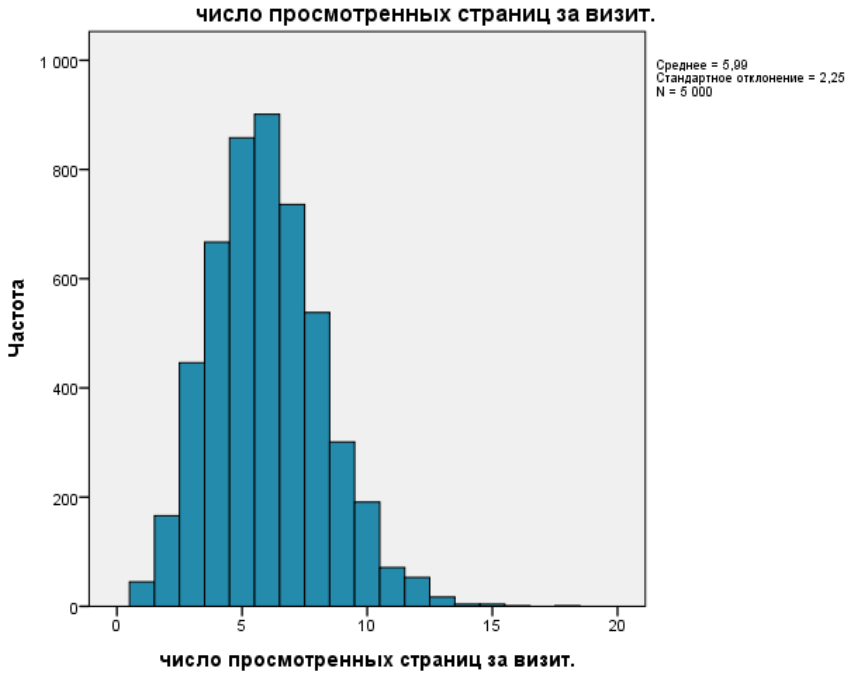
Статистики

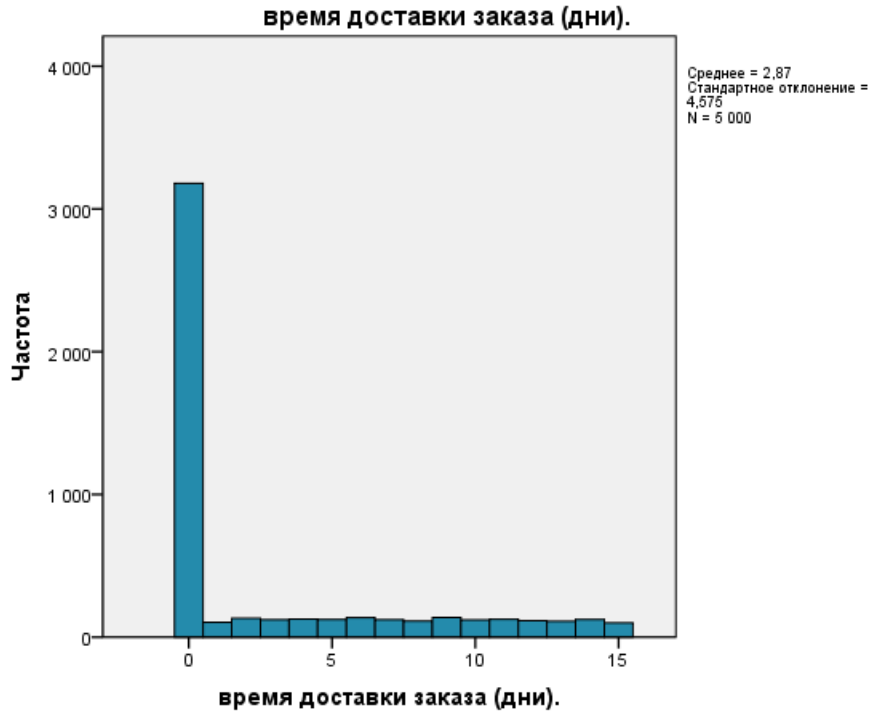
		сумма покупки	сколько секунд клиент провёл на сайте.	число просмотренных страниц за визит.	время доставки заказа (дни).	оценка удовлетворённости клиента.
N	Валидные	5000	5000	5000	5000	1820
	Пропущенные	0	0	0	0	3180
Среднее		34,9278	361,873	5,99	2,87	4,10
Медиана		,0000	305,450	6,00	,00	4,00
Стд. отклонение		87,26425	255,5043	2,250	4,575	,644
Дисперсия		7615,049	65282,459	5,064	20,928	,415
Минимум		,00	1,0	1	0	2
Максимум		1419,30	2235,3	18	15	5

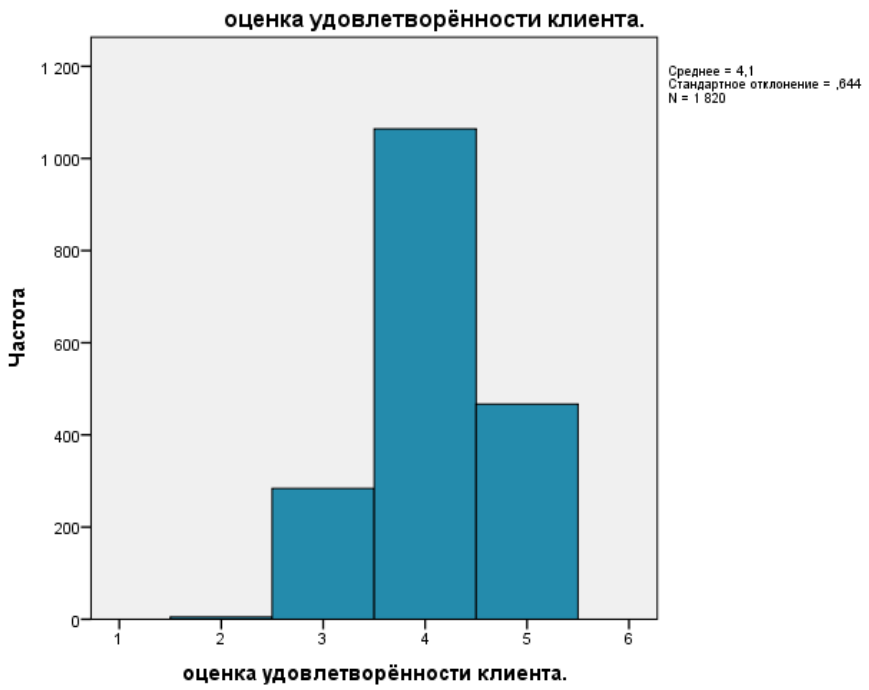
В SPSS также как и в Excel нет быстрого вывода коэффициента вариации, его придется рассчитать как отношение значения стандартного отклонения к среднему значению. Одновременно с таблицей сводной статистики, визуализируются гистограммы (с возможностью сравнения с кривой нормального распределения)


сумма покупки
Среднее = 34,93
Стандартное отклонение = 87,264
N = 5 000
Частота
4 000
3 000
2 000
1 000
0
,00
200,00
400,00
600,00
800,00
1000,00
1200,00
1400,00
сумма покупки


сколько секунд клиент провёл на сайте.
Среднее = 361,87
Стандартное отклонение = 255,504
N = 5 000
Частота
500
400
300
200
100
0
,0
500,0
1000,0
1500,0
2000,0
2500,0
сколько секунд клиент провёл на сайте.


число просмотренных страниц за визит.
Среднее = 5,99
Стандартное отклонение = 2,25
N = 5 000
Частота
1 000
800
600
400
200
0
0
5
10
15
20
число просмотренных страниц за визит.


время доставки заказа (дни).
Среднее = 2,87
Стандартное отклонение = 4,575
N = 5 000
Частота
4 000
3 000
2 000
1 000
0
0
5
10
15
время доставки заказа (дни).



Время для аналогичной работы в SPSS потребовалось существенно меньше чем в Excel. Все данные рассчитывались также как и в Excel на основе всей выборки без учета воронки данных. В дальнейшем при детальном описании экономической составляющей и выводов это будет учтено.

SQL

Визуализация в SQL (csvfiddle.io) невозможна, но представленные ниже запросы могут быть реализованы в Metabase с полноценной визуализацией данных.

Все запросы при частотном анализе строятся на основе одной и той же конструкции, меняется только столбец данных для группировки и расчета (для подготовки таблиц визуализации). Например частотный анализ по категориальной переменной “Источник трафика”

```
with freq as (
    select  traffic_source,
            count(*) as cnt
    from Dataset
    group by 1
)
select  *,
        round( cnt*100.0 / sum(cnt) over(),2) as percent
from freq
```

Результат

	traffic_source	cnt	percent
1	Organic	2199	43.98
2	PaidAds	1551	31.02
3	Referral	513	10.26
4	Email	737	14.74

Но для визуализации доли каждого вида трафика в общий результат в Metabase второй столбец даже избыточен. Абсолютно аналогично формируются частотные таблицы по количественным переменным, но с обязательной сортировкой по столбцу (order by). Также логично исключить нулевые значения по ключевым метрикам, чтобы оценить поведение посетителей, попавших в воронку, которые трансформировались в покупателей.

Пример для **время доставки заказа (для покупателей)**

```
with freq as (
    select  delivery_time_days,
            count(*) as cnt
    from Dataset
    where delivery_time_days is not null and  delivery_time_days !=0
    group by 1
    order by 1
)
select  *,
        round( cnt*100.0 / sum(cnt) over(),2) as percent
from freq
```

Результат

	delivery_time_days	cnt	percent
1	1	105	5.77
2	2	133	7.31
3	3	123	6.76
4	4	127	6.98
5	5	124	6.81
6	6	137	7.53
7	7	122	6.7
8	8	113	6.21
9	9	138	7.58
10	10	121	6.65
11	11	125	6.87
12	12	116	6.37
13	13	112	6.15
14	14	124	6.81
15	15	100	5.49

Для переменных time_on_site_s и purchase_amount, которые имееют большой разброс,для наглядной визуализации требуется агрегация по диапазонам (секунды для time_on_site_s и рубли для purchase_amount)

```
with freq as (
    select
        (floor(purchase_amount / 50) * 50) + 50 as amount,
        count(*) as cnt
    from dataset
    where purchase_amount is not null and purchase_amount != 0
    group by floor(purchase_amount / 50)
    order by amount
)
select  *,
        round( cnt*100.0 / sum(cnt) over(),2) as percent
from freq
```

Результат

	amount	cnt	percent
1	50	861	47.31
2	100	492	27.03
3	150	173	9.51
4	200	90	4.95
5	250	41	2.25
6	300	26	1.43
7	350	32	1.76
8	400	23	1.26
9	450	32	1.76
10	500	16	0.88

Сводная таблица статистики по показателям

```
select
    'Среднее' as статистика,
    round(avg(purchase_amount)*1.0, 3) as "сумма покупки",
    round(avg(time_on_site_s)*1.0, 2) as "время проведенное на сайте,
с",
    round(avg(pages_visited)*1.0, 2) as "просмотренные страницы",
    round(avg(delivery_time_days)*1.0, 2) as "время доставки",
    round(avg(satisfaction)*1.0, 2) as "удовлетворенность"
from dataset
union all
select
    'Медиана',
    round(percentile_cont(0.5) within group (order by
purchase_amount)*1.0, 2),
    round(percentile_cont(0.5) within group (order by
time_on_site_s)*1.0, 2),
    round(percentile_cont(0.5) within group (order by
pages_visited)*1.0, 2),
    round(percentile_cont(0.5) within group (order by
```

```
delivery_time_days)*1.0, 2),
    round(percentile_cont(0.5) within group (order by satisfaction)*1.0,
2)
from dataset
union all
select
    'Дисперсия',
    round(var_pop(purchase_amount)*1.0, 2),
    round(var_pop(time_on_site_s)*1.0, 2),
    round(var_pop(pages_visited)*1.0, 2),
    round(var_pop(delivery_time_days)*1.0, 2),
    round(var_pop(satisfaction)*1.0, 2)
from dataset
union all
select
    'Коэф. вариации',
    round((stddev_pop(purchase_amount) / nullif(avg(purchase_amount),0
)) * 1.0, 4),
    round((stddev_pop(time_on_site_s) / nullif(avg(time_on_site_s),0 ))
* 1.0, 4),
    round((stddev_pop(pages_visited) / nullif(avg(pages_visited), 0)) *
1.0, 4),
    round((stddev_pop(delivery_time_days) /
nullif(avg(delivery_time_days), 0)) * 1.0, 4),
    round((stddev_pop(satisfaction) / nullif(avg(satisfaction), 0)) *
1.0, 4)
from dataset
```

Результат

	статистика	сумма покупки	время проведенн...	просмотренные с...	время доставки	удовлетворенность
1	Среднее	34.928	361.87	5.99	2.87	4.1
2	Медиана	0	305.45	6	0	4
3	Дисперсия	7613.53	65269.4	5.06	20.92	0.41
4	Коэф. вариации	2.4982	0.706	0.3757	1.5928	0.1572

Написание кода по затратам времени было меньше чем при написанием формул в Excel и работы со сводными таблицами, но больше чем использования готовых шаблонов SPSS

Python

На начальном этапе пришлось долго подгонять типы данных для корректного парсинга, на втором этапе для корректного отображения гистограмм пришлось корректировать параметр bins, так как для переменных `pages_visited` и `satisfaction` диапазон значений небольшой и на графике разделение оси OX проводилось не на целые числа, а с шагом 0,5. Это вызывало некорректное отображение. В EXcel происходило тоже самое и приходилось подстраиваться, самая быстрая и корректная визуализация распределения реализована в SPSS.

Итоговый результат по гистограммам: Конфигурация для графиков по каждой переменной, цвета + подписи

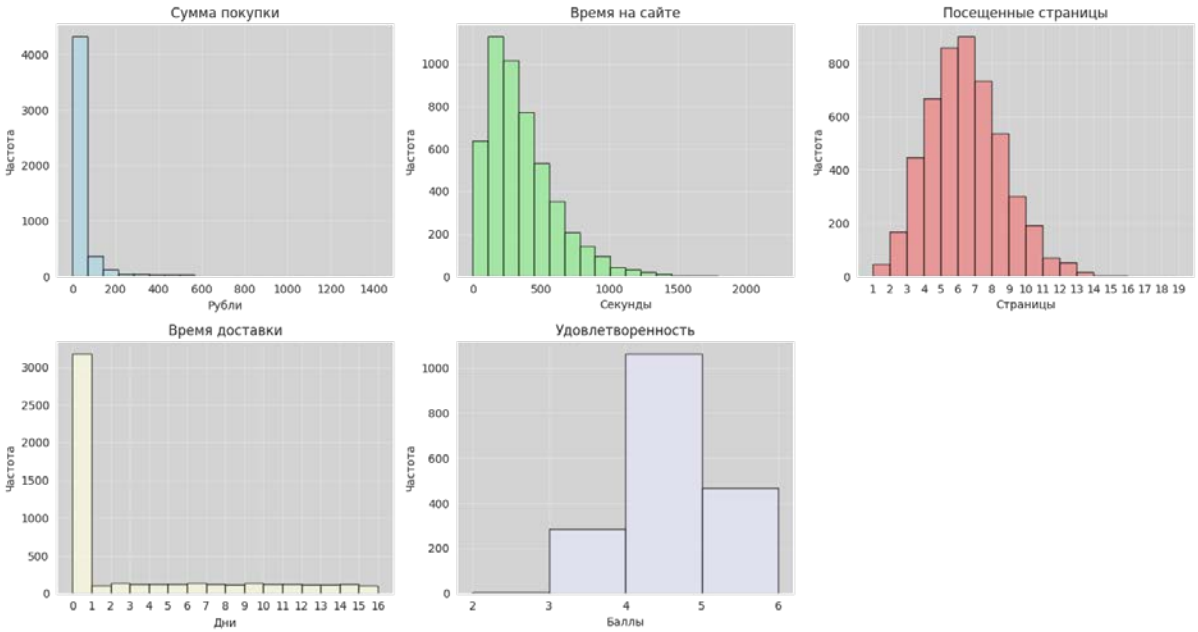
```
import matplotlib.pyplot as plt

# Данные
data_list = [
    ecom['purchase_amount'],
    ecom['time_on_site_s'],
    ecom['pages_visited'],
    ecom['delivery_time_days'],
    ecom['satisfaction']
]

titles = ['Сумма покупки', 'Время на сайте', 'Посещенные страницы',
          'Время доставки', 'Удовлетворенность']
xlabels = ['Рубли', 'Секунды', 'Страницы', 'Дни', 'Баллы']
colors = ['lightblue', 'lightgreen', 'lightcoral', 'lightyellow', 'lavender']
```

Строим гистограммы для ключевых переменных (*purchase_amount, time_on_site, pages_visited, delivery_time, satisfaction*) с персонифицированными настройками для переменных, содержащих диапазон менее 20 значений

```
# Создаем графики
fig, axes = plt.subplots(2, 3, figsize=(15, 8), facecolor='none')
axes_flat = axes.flatten()

for i in range(5):
    ax = axes_flat[i]
    ax.set_facecolor('lightgray')

    #настраиваем количество точек по оси OX для корректного отображения
    #целочисленных переменных с диапазоном меньше 20
    data_min = int(data_list[i].min())
    data_max = int(data_list[i].max())

    # Устанавливаем тики на оси X -- целые значения от data_min до data_max
    if (data_max - data_min + 1) < 20:
      # Устанавливаем персональные бины и тики на оси X -- от data_min до
data_max
      bins_x = range(data_min, data_max+2)
      ax.set_xticks(range(data_min, data_max + 2))
    else:
      bins_x = 20

    # Простая гистограмма
```

```
    ax.hist(data_list[i].dropna(),
            color=colors[i],
            edgecolor='black',
            bins = bins_x, #выставляем бины
            alpha=0.7)

    ax.set_title(titles[i])
    ax.set_xlabel(xlabels[i])
    ax.set_ylabel('Частота')
    ax.grid(True, alpha=0.3)
    #Отображение только целых значений
    ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"{int(x)}"))

axes_flat[5].axis('off')
plt.tight_layout()
plt.show()
```

Результат- все графики одним блоком.



Если в дальнейшем этот код использовать как шаблон, то анализ будет очень быстрый, но при первом написании с учетом что это делается первый раз, а также нюансами отображения целых чисел на оси OX затраты времени были колоссальны.

Для расчета статистик - среднее, медиана, дисперсия, коэффициент вариации сначала проведем расчет на основе все выборки, Затем исключим нулевые данных, характерные для тех пользователей, кто не совершал покупок (наиболее очевидно, на примере переменной delivery_time_days)

Код

```python
import numpy as np

# Столбцы для анализа
columns_to_analyze = ['purchase_amount', 'time_on_site_s',
                      'pages_visited', 'delivery_time_days', 'satisfaction']

# Функция для расчета статистик
def calculate_stats(series):
    data = series.dropna() #исключили NaN
    if data.empty:
        return pd.Series([np.nan, np.nan, np.nan, np.nan],
                         index=['Среднее', 'Медиана', 'Дисперсия', 'Коэф. вариации'])

    mean_val = data.mean()
    median_val = data.median()
    variance_val = data.var()
    cv_val = (data.std() / mean_val) if mean_val != 0 else np.nan

    return pd.Series([mean_val, median_val, variance_val, cv_val],
                     index=['Среднее', 'Медиана', 'Дисперсия', 'Коэф. вариации'])

# Применяем функцию ко всем столбцам сразу
results = ecom[columns_to_analyze].apply(calculate_stats).T

# Форматирование в один проход
format_func = lambda x: f'{x:.2f}' if pd.notna(x) else 'NaN'
formatted_results = results.copy()
formatted_results['Среднее'] = formatted_results['Среднее'].apply(format_func)
formatted_results['Медиана'] = formatted_results['Медиана'].apply(format_func)
formatted_results['Дисперсия'] = formatted_results['Дисперсия'].apply(format_func)
formatted_results['Коэф. вариации'] = (
    formatted_results['Коэф. вариации'].apply(
        lambda x: f'{x:.2f}' if pd.notna(x) else 'NaN'
    )
)

print("Статистики:")
print("=" * 80)
print(formatted_results.to_string())
print("=" * 80)
```

Результат

```
Статистики по столбцам:
==========================================================================
                    Среднее Медиана Дисперсия Коэф. вариации
purchase_amount       34.93    0.00   7615.05          2.50
time_on_site_s       361.87  305.45  65282.46          0.71
pages_visited          5.99    6.00      5.06          0.38
delivery_time_days     2.87    0.00     20.93          1.59
satisfaction           4.10    4.00      0.41          0.16
==========================================================================
```

Исключим нулевые значения из массива данных и повторим расчет функция `calculate_stats` доработана

```
# Функция для расчета статистик #############
# Использовать в будущем  ###################
def calculate_stats(series, zero_free=False):
    """
    Вычисляет базовые статистики для серии данных.
    Параметры:
    - series: pd.Series -- входные данные;
    - zero_free: bool -- если True, заменяет 0 на NaN перед расчётом.
    Возвращает:
    - pd.Series с показателями: Среднее, Медиана, Дисперсия, Коэф. вариации.
    """
    # Если zero_free=True, заменяем нули на NaN
    if zero_free:
        series = series.replace(0, np.nan)

    # Исключаем NaN (включая потенциально добавленные нули)
    data = series.dropna()

    if data.empty:
        return pd.Series([np.nan, np.nan, np.nan, np.nan],
                         index=['Среднее', 'Медиана', 'Дисперсия', 'Коэф. вариации'])

    mean_val = data.mean()
    median_val = data.median()
    variance_val = data.var()
    cv_val = (data.std() / mean_val) if mean_val != 0 else np.nan

    return pd.Series([mean_val, median_val, variance_val, cv_val],
                     index=['Среднее', 'Медиана', 'Дисперсия', 'Коэф. вариации'])

#############################################
```

Вызов функции `results = ecom[columns_to_analyze].apply(lambda s: calculate_stats(s, zero_free=True)).T`

и дальнейшее форматирование по аналогии позволило получить следующие результаты:

```
Статистики:
```

```
=====================================================================
                    Среднее Медиана Дисперсия Коэф. вариации
purchase_amount       95.96   52.88  15068.63           1.28
time_on_site_s       361.87  305.45  65282.46           0.71
pages_visited          5.99    6.00      5.06           0.38
delivery_time_days     7.89    8.00     17.91           0.54
satisfaction           4.10    4.00      0.41           0.16
=====================================================================
```

B. Формирование выборок

Реализация дальнейших пунктов с применением Excel, SPSS, SQL становится нецелесообразной, так как эти специфические задачи потребуют бОльших затрат чем написание кода на Python.

Формируем выборки (три датасет) -simple_sample / strat_sample / systematic_sample

```
# Простая случайная выборки
simple_sample = ecom.sample(n=800, random_state=42)

# Стратифицированная выборка (по region)
from sklearn.model_selection import train_test_split

_, strat_sample = train_test_split(   #выгружаем в кортеж
    ecom,
    test_size=800,
    random_state=42,
    stratify=ecom['region']
)

# Систематическая выборка
N = len(ecom)
k = np.floor(N / 800).astype(int)
start_idx = np.random.randint(0, k)
systematic_sample = ecom.iloc[start_idx::k].head(800)
```

Для выполнения второй части задания *“Для каждой выборки рассчитайте среднее purchase_amount, стандартное отклонение, конверсию и медиану. Сравните с генеральной совокупностью.”* воспользуемся ранее написанной функцией `calculate_stats` дополнив её расчетом стандартного отклонения и величиной выборки N. Навреное, это быо бы логичнее прогнать как-то в цикле, но я решил оставить линейный подход для большей наглядности. Рассмотрим как ведут себя статистические характеристики каждой выборки в сравнении с генеральной совокупностью.

Код

```
#Расчет статистики по ГС
result = calculate_stats(ecom['purchase_amount'] )
#Для конверсии исключим нулевые значения purchase_amount
result_client = calculate_stats(ecom['purchase_amount'],True )
conv = result_client['N'] /result['N']

#Расчет статистики для простой случайной выборки
res_simple = calculate_stats(simple_sample['purchase_amount'] )
res_simple_client = calculate_stats(simple_sample['purchase_amount'], True )
conv_simple = res_simple_client['N'] / res_simple['N']

#Расчет статистики для стратифицированной выборки
res_strat = calculate_stats(strat_sample['purchase_amount'] )
res_strat_client = calculate_stats(strat_sample['purchase_amount'], True )
conv_strat = res_strat_client['N'] / res_strat['N']

#Рассчет статистики для систематической выборки
res_systematic = calculate_stats(systematic_sample['purchase_amount'] )
res_systematic_client = calculate_stats(systematic_sample['purchase_amount'], True )
conv_systematic = res_systematic_client['N'] / res_systematic['N']

#Выыод данных
print (f"Генеральная совокупность\n\n{result.to_string()}")
print (f"Конверсия\t\t{conv:.2%}")
print ()

print (f"Простая случайная\n\n{res_simple.to_string()}")
print (f"Конверсия\t\t{conv_simple:.2%}")
print ()

print (f"Стратифицированная\n\n{res_strat.to_string()}")
print (f"Конверсия\t\t{conv_strat:.2%}")
print ()

print (f"Систематизированная\n\n{res_systematic.to_string()}")
print (f"Конверсия\t\t{conv_systematic:.2%}")
print ()
```

Наиболее точно описывает генеральную совокупность систематизированная выборка.

Но по показателю конверсии лучшая - стратифицировнная. Это обусловлено, тем что при стратифицированной выборке исключены позиции где не было рекламных компаний, а это составляет 2167 позиций без продаж 43% от всей выборки. То есть на стадии выборки мы условно максимизируем попадание в покупателя, а не просто в посетителя сайта. Речь идет о конверсии в первую очередь, но так как нулевых продаж 3180 (63,6%) то разумеется средние величины “пляшут” а медиана везде равна 0

Есть продажи	Была компания	538
Есть продажи	Не было компаний	1282
Нет продаж	Была компания	1013
Нет продаж	Не было компаний	2167

Результат:

Генеральная совокупность		Простая случайная	
N	**5000.000000**	N	800.000000
Среднее	**34.927770**	Среднее	29.782212
Медиана	**0.000000**	Медиана	0.000000
Дисперсия	**7615.048930**	Дисперсия	5408.669144
Станд.отклонение	**87.264248**	Станд.отклонение	73.543655
Коэф. вариации	**2.498420**	Коэф. вариации	2.469382
Конверсия	**36.40%**	Конверсия	33.88%
Стратифицированная		**Систематизированная**	
N	800.000000	**N**	**800.000000**
Среднее	30.265350	**Среднее**	**34.967400**
Медиана	0.000000	**Медиана**	**0.000000**
Дисперсия	5222.987748	**Дисперсия**	**8620.312876**
Станд.отклонение	72.270241	**Станд.отклонение**	**92.845640**
Коэф. вариации	2.387887	**Коэф. вариации**	**2.655206**
Конверсия	36.25%	**Конверсия**	**35.75%**

C. Количественная репрезентативность

Наибольшая репрезентативность у систематизированной выборки

код

```
#Решаем в лоб без циклов (пока)
dev_for =[]

# для Простая случайная - 0
abs_dev =  abs(res_simple['Среднее'] - result ['Среднее'])
rel_dev = abs_dev /result ['Среднее']
dev_for.append(('простая случайная', abs_dev,rel_dev) )

# для Стратифитцированная - 1
abs_dev =  abs(res_strat['Среднее'] - result ['Среднее'])
rel_dev = abs_dev /result ['Среднее']
dev_for.append(('стратифитцированная', abs_dev,rel_dev) )

# для Систематизированная - 2
abs_dev =  abs(res_systematic['Среднее'] - result ['Среднее'])
rel_dev = abs_dev /result ['Среднее']
dev_for.append(('систематизированная', abs_dev,rel_dev) )

min_element = min(dev_for, key=lambda x: x[2])  #прикольная фича
print(f"Наиболее репрезентативна {min_element[0]} выборка")
```

D. Доверительные интервалы

Результат:

```
95% доверительный интервал для среднего:
для выборки - ген. совокупность:        (32.51, 37.35)       дина интервала: 4.839
для выборки - простая случайная :       (24.68, 34.89)       дина интервала: 10.21
для выборки - стратифицированная:       (25.25, 35.28)       дина интервала: 10.03
для выборки - систематизированная:      (29.42, 40.82)       дина интервала: 11.39
```

Показывает, что ожидаемо самый узкий интервал у генеральной совокупности (ширина ≈ 4.84). Выборочные интервалы шире (ширина ≈ 10-11.4), что показывает неопределённость из-за меньшего объёма данных. Стратифицированная выборка точнее простой случайной (уже на ≈ 0.4). Систематическая выборка имеет самый широкий интервал (≈ 11.4) — наименее точная, возможно, уловила нерепрезентативный паттерн.

Все интервалы перекрываются, значит различия между выборками статистически незначимы, но стратифицированная выборка показала себя лучше других методов выборки для оценки среднего.

код

```
# 95% доверительный интервал
dov_interv = []

#ГС
n = result ['N']
mean_val = result ['Среднее']
std_val = result ['Станд.отклонение']  # несмещенная оценка

dov_interv.append(('ген. совокупность', n, mean_val,std_val) )

# для Простая случайная
n = res_simple ['N']
mean_val = res_simple ['Среднее']
std_val = res_simple ['Станд.отклонение']

dov_interv.append(('простая случайная', n, mean_val,std_val) )
# для Стратифитцированная
n = res_strat ['N']
mean_val = res_strat ['Среднее']
std_val = res_strat ['Станд.отклонение']

dov_interv.append(('стратифицированная', n, mean_val,std_val) )
# для Систематизированная
n = res_systematic ['N']
mean_val = res_systematic ['Среднее']
std_val = res_systematic ['Станд.отклонение']

dov_interv.append(('систематизированная', n, mean_val,std_val) )
```

```
# прогоним в цикле для всех ( а ведь было проще написать одну функцию,
# чтобы закинуть тудв пандас и на выходе интервал, но уже сделал)
print(f"95% доверительный интервал для среднего:")
for item in dov_interv:
    s = item[0]           # строка
    n = int(item[1])     # преобразуем np.float64 в обычный float
    mean_val = float(item[2])
    std_val = float(item[3])

    alpha = 0.05
    t_critical = stats.t.ppf(1 - alpha/2, df=n-1)
    margin_of_error = t_critical * (std_val / np.sqrt(n))

    ci_lower = mean_val - margin_of_error
    ci_upper = mean_val + margin_of_error
    l= ci_upper - ci_lower
    print(f"для выборки - {s}: \t({ci_lower:.2f}, {ci_upper:.2f})\t дина интервала:
{l:.4}")
```

Для построения доверительных интервалов для конверсии видимо сначала надо из генеральной совокупности многократно (1000 раз) взять выборку по 800 человек и в каждой выборке посчитать конверсию. Потому что в отличии от суммы покупки - характеристика каждой строки в начальном датасете. Конверсия - это одно число , рассчитанное по всему датасету - отношение количества посетителей с purchase_amount >0 к общему количеству. Как в пункте E, по крайне мере я так понял.

```
n_iterations = 1000
sample_size = 800

conversion = pd.Series([
    (population.sample(n=sample_size, replace=True) > 0).mean()
    for _ in range(n_iterations)
])

#Повторяем что было, хотя можно было это сделать сразу и загнать в цикл выше
conv_res = calculate_stats(conversion)

n = conv_res ['N']
mean_val = conv_res ['Среднее']
std_val = conv_res ['Станд.отклонение']
s = 'Конверсия'

alpha = 0.05
t_critical = stats.t.ppf(1 - alpha/2, df=n-1)
margin_of_error = t_critical * (std_val / np.sqrt(n))

ci_lower = mean_val - margin_of_error
ci_upper = mean_val + margin_of_error
l= ci_upper - ci_lower
print(f"для выборки - {s}: \t({ci_lower:.2f}, {ci_upper:.2f})\t дина интервала: {l:.4}")
```

В полном соответствии с законом больших чисел - для конверсии доверительный интервал “неприлично” маленький. (0.36, 0.37) дина интервала: 0.002179
А потом повторил этот же расчет по формуле из лекции - результат практически сошелся: (0.35, 0.38) дина интервала: 0.002267

код

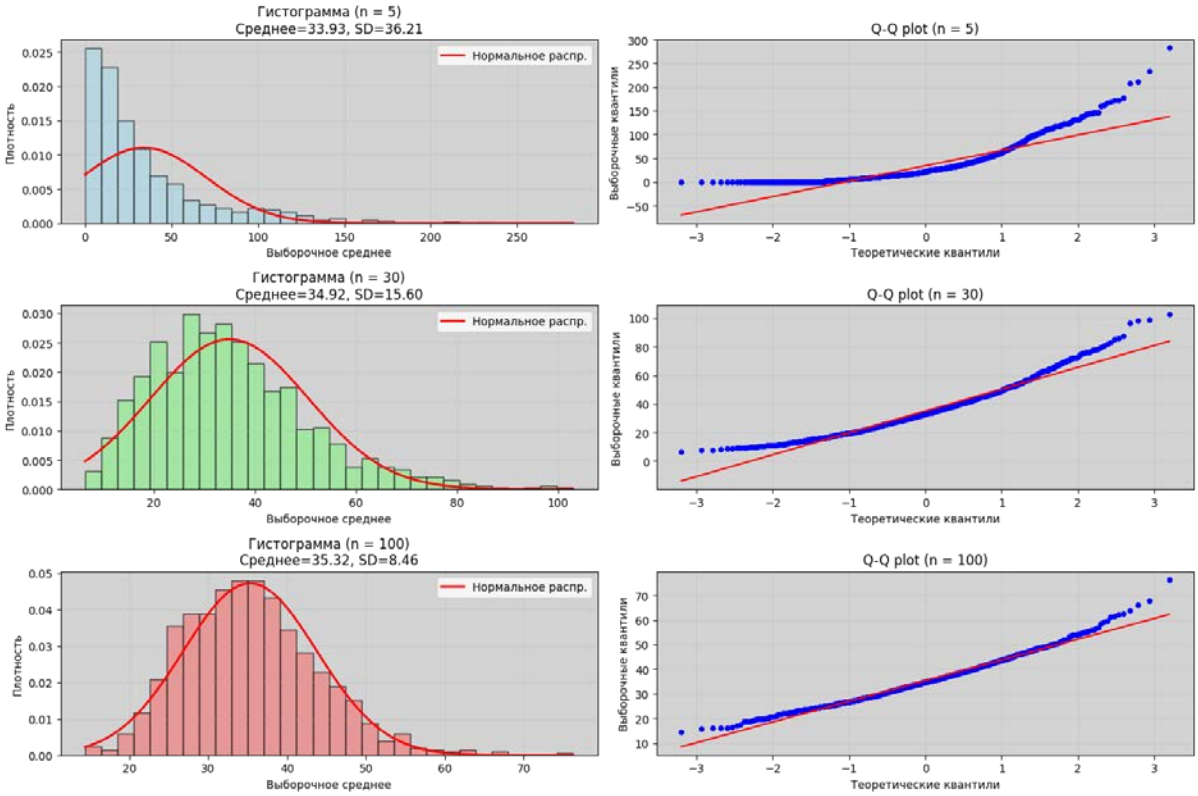
```
from math import sqrt

n = len(population)
m = (population > 0).sum()

p_hat = m / n              # Рассчитать выборочную долю
q_hat  = 1 - p_hat

z = 1.96                   #Для ДИ 95% z ≈ 1.96 (из таблицы)

se = sqrt((p_hat * q_hat) / n)  # Стандартная ошибка

ci_lower = p_hat - z * se  # Нижняя граница ДИ
ci_upper = p_hat + z * se  # Верхняя граница ДИ

print(f"для генеральной совокупности: \t({ci_lower:.2f}, {ci_upper:.2f})\t дина
интервала: {l:.4}"
```

E. ЦПТ и Закон больших чисел

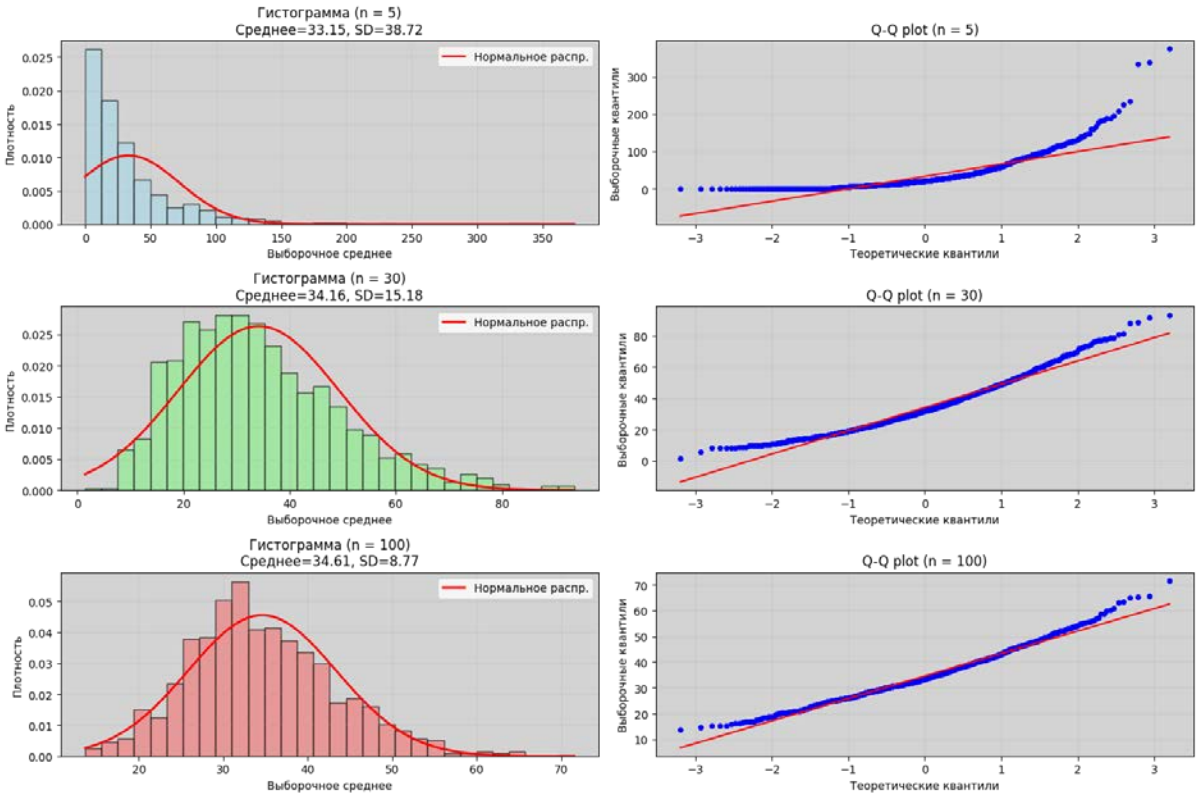
Выводы в полной мере соответствуют теоретическим основам ЦПТ. При увеличении величины подвыборки средние величины становятся более точными (высота и острота колокола нормального распределения увеличивается с 0,01 - при n =5 до 0,045 при n = 100 (более чем 4 раза) и сама величина среднего из средних становится близка с среднему ГС при n=30 (первый прогон, но это скорее случайность, так как в ГС избыточно много нулевых значений)
ср. по ГС = **34.927770 (рассчитано в пункте B)**
n=5, ср = 33.93 / 33.15
n=30, ср = 34.92 / 34.16
n=100, ср = 35.32 / 34.61

Особенно наглядно это видно из графика Q-Q plot

Первый запуск



Второй запуск



Код

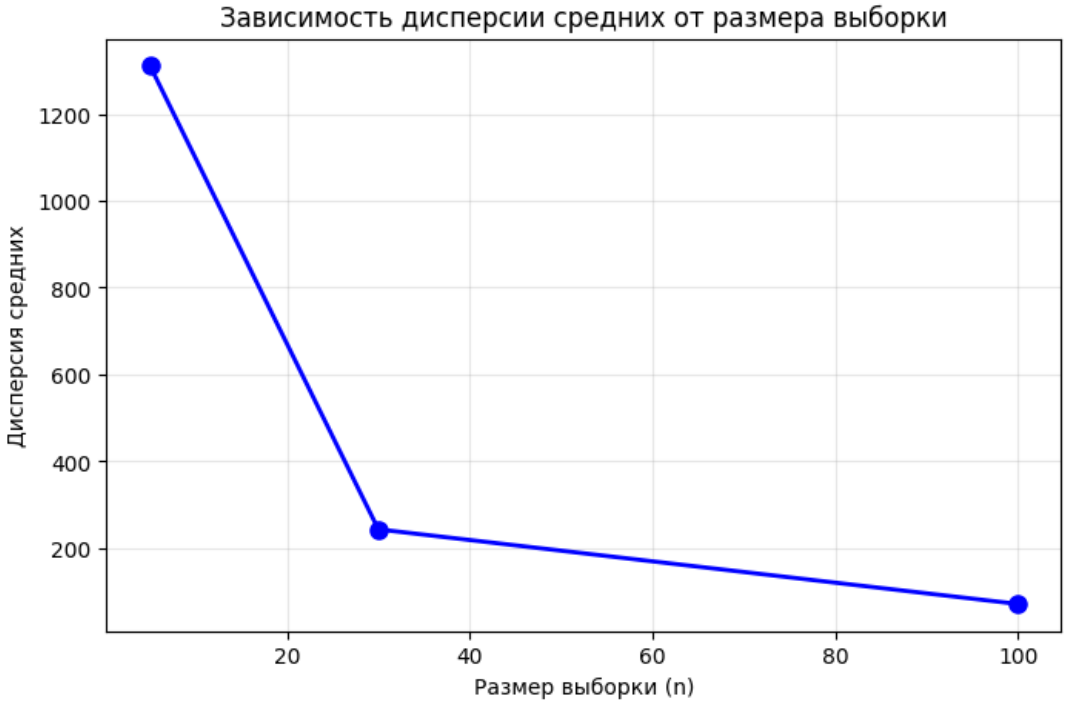
```
population = ecom['purchase_amount']
```

```
sample_sizes = [5, 30, 100]
n_iterations = 1000
colors = ['lightblue', 'lightgreen', 'lightcoral']

# Создаем графики
fig, axes = plt.subplots(3, 2, figsize=(15, 10), facecolor='none')
axes = axes.flatten()

variances = [] # для дисперсии

for idx, n in enumerate(sample_sizes):
    # Сбор средних
    sample_means = []
    for _ in range(n_iterations):
        sample = np.random.choice(population, size=n, replace=True)
        sample_means.append(np.mean(sample))

    sample_means = np.array(sample_means) #массив средних
    variances.append(sample_means.var())  #дисперсия

    # Гистограмма (левый столбец)
    ax_hist = axes[idx*2]
    ax_hist.set_facecolor('lightgray')

    # Настройка бинов и тиков для целых значений если диапазон маленький как в части А
    data_min = int(sample_means.min())
    data_max = int(sample_means.max())

    if (data_max - data_min + 1) < 30:
        bins_x = range(data_min, data_max+2)
        ax_hist.set_xticks(range(data_min, data_max + 2))
    else:
        bins_x = 30

    ax_hist.hist(sample_means,
                 bins=bins_x,
                 color=colors[idx],
                 edgecolor='black',
                 alpha=0.7,
                 density=True)

    # Теоретическая нормальная кривая
    x = np.linspace(sample_means.min(), sample_means.max(), 100)
    normal_pdf = stats.norm.pdf(x, loc=sample_means.mean(), scale=sample_means.std())
    ax_hist.plot(x, normal_pdf, 'r-', linewidth=2, label='Нормальное распр.')

    ax_hist.set_title(f'Гистограмма (n = {n})\nСреднее={sample_means.mean():.2f},
SD={sample_means.std():.2f}')
    ax_hist.set_xlabel('Выборочное среднее')
    ax_hist.set_ylabel('Плотность')
    ax_hist.grid(True, alpha=0.3)
    ax_hist.legend()

    # Q-Q plot (правый столбец)
    ax_qq = axes[idx*2 + 1]
    ax_qq.set_facecolor('lightgray')
```

```
    stats.probplot(sample_means, dist="norm", plot=ax_qq)

    # Стилизация Q-Q plot
    ax_qq.get_lines()[0].set_markersize(4)
    ax_qq.get_lines()[1].set_color('red')
    ax_qq.set_title(f'Q-Q plot (n = {n})')
    ax_qq.set_xlabel('Теоретические квантили')
    ax_qq.set_ylabel('Выборочные квантили')
    ax_qq.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Дисперсия средних значений с ростом величины выборки уменьшается:

```
При n: 5     дисперсия: 1311.256
При n: 30    дисперсия: 243.453
При n: 100   дисперсия: 71.592
```

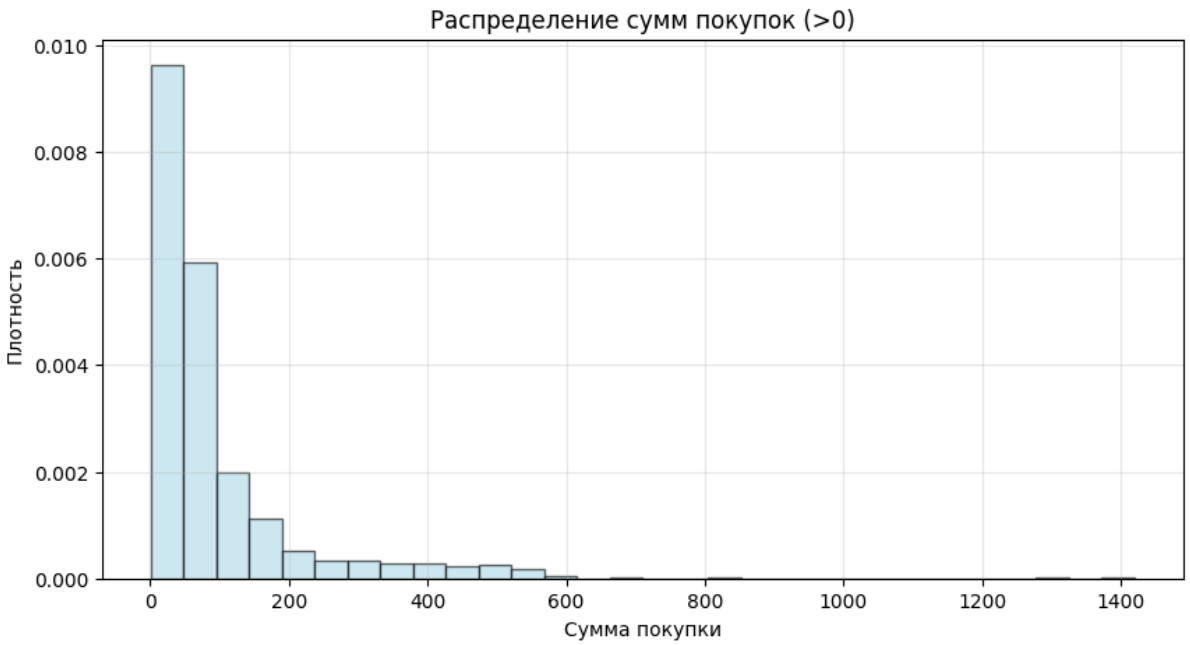


Код

```
for idx, n in enumerate(sample_sizes):
  print(f"При n: {n}\t дисперсия: {variances[idx]:.3f}")

plt.figure(figsize=(8, 5))
plt.plot(sample_sizes, variances, 'bo-', linewidth=2, markersize=8)
plt.xlabel('Размер выборки (n)')
plt.ylabel('Дисперсия средних')
plt.title('Зависимость дисперсии средних от размера выборки')
plt.grid(True, alpha=0.3)
plt.show()
```

F. Параметрические оценки

Распределение суммы покупки у покупателей (purchase_amount >0) представлены на гистограмме.



Код

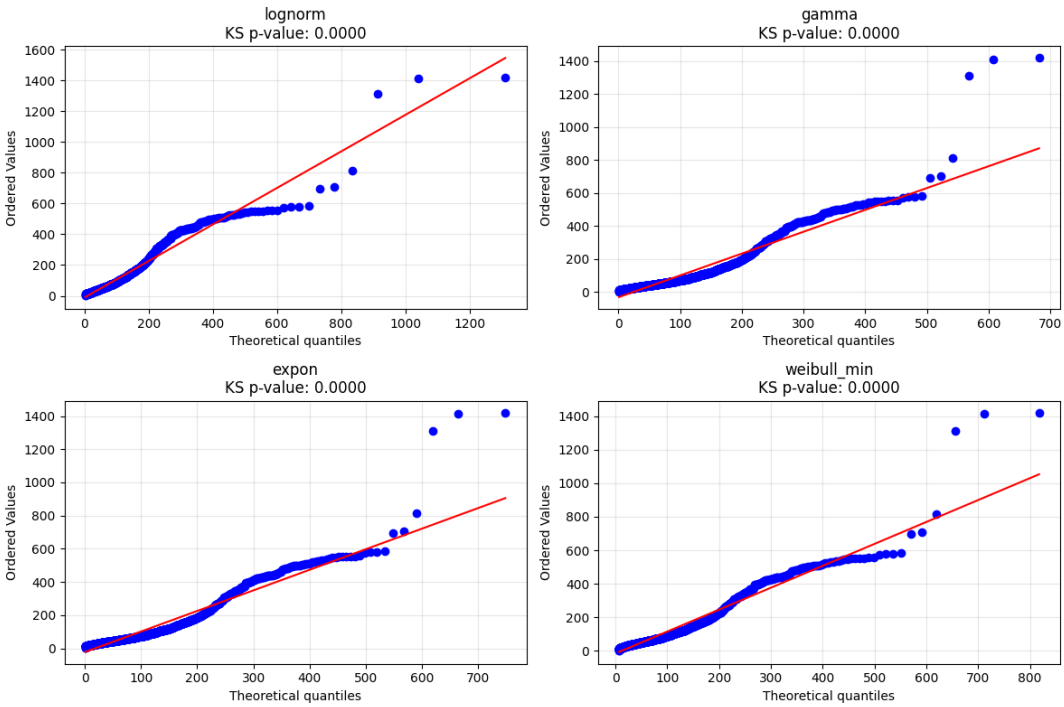
```
data = ecom[ecom['purchase_amount'] > 0]['purchase_amount'].values

# Гистограмма данных
plt.figure(figsize=(10, 5))

plt.hist( data,
          bins=30,
          density=True,
          alpha=0.6,
          color='lightblue',
          edgecolor='black')

plt.xlabel('Сумма покупки')
plt.ylabel('Плотность')
plt.title('Распределение сумм покупок (>0)')
plt.grid(alpha=0.3)
plt.show()
```

Создадим 4 типовых распределения - лог.нормальное, гамма, экспоненциальное и распределение Вейбулла

```
distributions = [
    ('lognorm', stats.lognorm),       #Лог нормальное
    ('gamma', stats.gamma),           #Гамма
    ('expon', stats.expon),           #Экспоненциальное
    ('weibull_min', stats.weibull_min)#Распределение Вейбулла
]
```

```
# Подгонка методом максимального правдоподобия
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()
```

Проверку подгонки проверяем визуально по Q-Q pots + тест Колмогорова-Смирнова о проверке гипотезы о принадлежности данных к конкретному распределению. При принадлежности p-value (значимость) должна быть >0.05



КОД

```
results = []
for idx, (dist_name, dist) in enumerate(distributions):
    ax = axes[idx]

    params = dist.fit(data)
    fitted_dist = dist(*params)

    # QQ-plot для проверки
    stats.probplot(data, dist=fitted_dist, plot=ax)

    # Тест Колмогорова-Смирнова
    ks_stat, ks_pvalue = stats.kstest(data, fitted_dist.cdf)

    results.append({
        'distribution': dist_name,
        'params': params,
        'ks_statistic': ks_stat,
        'ks_pvalue': ks_pvalue,
         'aic': 2 * len(params) - 2 * fitted_dist.logpdf(data).sum()
        })
          ax.set_title(f'{dist_name}\nKS p-value: {ks_pvalue:.4f}')
    ax.grid(alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

Результаты подгонки распределений:

```
  distribution  ks_pvalue         aic
0      lognorm        0.0  19724.4445
1        gamma        0.0  20195.2582
2        expon        0.0  20215.7187
3  weibull_min        0.0         inf
```

p-value < 0.01 у всех → ни одно распределение не подходит идеально (тест Колмогорова-Смирнова провален)

Критерии выбора:

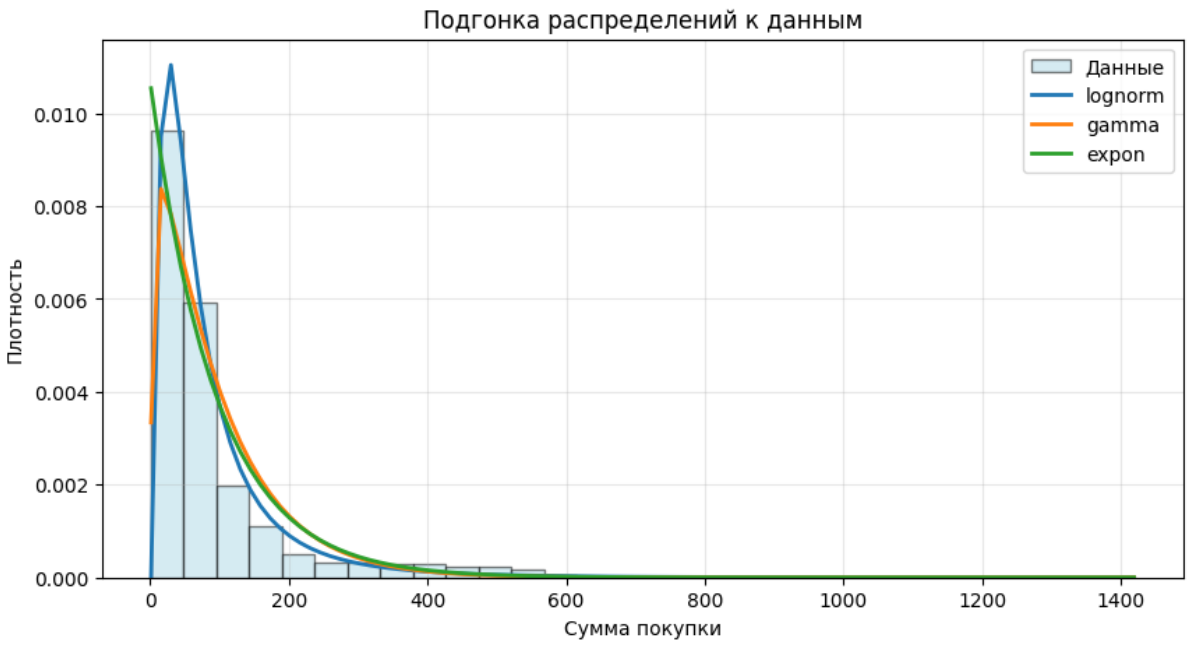
1. AIC меньше → лучше: lognorm (19724) < gamma (20195) < expon (20216)
2. Вейбулл inf → плохая сходимость, отбрасываем
3. Q-Q plot: Гамма лучше в центре, но логнорм лучше описывает хвосты
4. Логнорм часто используется для финансовых данных

Вывод: Логнорм — "наименее плохое" распределение, но не идеальное.

Код

```
results_df = pd.DataFrame(results)
print("\nРезультаты подгонки распределений:")
print(results_df[['distribution', 'ks_pvalue', 'aic']].round(4))

# Визуализация лучшего распределения
plt.figure(figsize=(10, 5))
x = np.linspace(data.min(), data.max(), 100)

# Гистограмма данных
plt.hist(data, bins=30, density=True, alpha=0.5, label='Данные', color='lightblue',
edgecolor='black')

# Подогнанные распределения
for dist_name, dist in distributions[:3]:  # первые три
    params = dist.fit(data)
    fitted_dist = dist(*params)
    plt.plot(x, fitted_dist.pdf(x), label=dist_name, linewidth=2)

plt.xlabel('Сумма покупки')
plt.ylabel('Плотность')
plt.title('Подгонка распределений к данным')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Визуализация выбранного “лучшим” распределением подтверждает выбор, так как визуально более точно повторяет контуры гистограммы, следовательно лог.нормальное распределение можно использовать для симуляций/прогнозов.



G. Свойства оценок и обсуждение

Решаем по аналогии с задачей E
создаем массив из подвыборок (если так можно сказать) n = 10, 30, 50, 100, 200

Для каждой подвыборки рассчитаем статистические показатели используя созданную ранее функцию calculate_stats. Посчитаем количество возрастающих и убывающих расхождений со статистикой ГМС (среднее, дисперсия, стандартное отклонение)

код

```
sample_sizes = [10, 30, 50, 100, 200]  #переопределим массив
samples = {}

stats_all = [] #спсиок из пандас датафреймов для хранения статистк выборок

# Создаем 5 выборок разных размеров
for size in sample_sizes:
      key = f'sample_{size}'
      samples[key] = np.random.choice(population, size=size, replace=True)
      stats_all.append(calculate_stats(pd.Series(samples[key])) )

# Инициализируем переменные для оценки расхождений
counters = {
    'mean': {'inc': 0, 'dec': 0},
    'variance': {'inc': 0, 'dec': 0},
    'std': {'inc': 0, 'dec': 0}
}
```

```
prev_deviations = {'mean': None, 'variance': None, 'std': None}
all_deviations = [] #величина отклонений

# Основной цикл
for i,stats in enumerate(stats_all):
    # Текущие отклонения
    deviations = {
        'sample_size': sample_sizes[i],
        'mean': stats['Среднее'] - result['Среднее'],
        'variance': stats['Дисперсия'] - result['Дисперсия'],
        'std': stats['Станд.отклонение'] - result['Станд.отклонение']
    }
    all_deviations.append(deviations)

    # Проверяем каждую метрику
    for metric in ['mean', 'variance', 'std']:
        if prev_deviations[metric] is not None:

           current_abs = abs(deviations[metric])
           prev_abs = abs(prev_deviations[metric])

           if current_abs > prev_abs:
                counters[metric]['inc'] += 1
           elif current_abs < prev_abs:
                counters[metric]['dec'] += 1

    # Обновляем предыдущие значения
    prev_deviations = deviations

# Выводим результаты
print("Среднее: возрастающих =", counters['mean']['inc'], "убывающих =",
counters['mean']['dec'])
print("Дисперсия: возрастающих =", counters['variance']['inc'], "убывающих =",
counters['variance']['dec'])
print("Стандартное отклонение: возрастающих =", counters['std']['inc'], "убывающих =",
counters['std']['dec'])
```

результат

```
Среднее: возрастающих = 1 убывающих = 3
Дисперсия: возрастающих = 1 убывающих = 3
Стандартное отклонение: возрастающих = 1 убывающих = 3
```

Если убывающих больше чем возрастающих по всем метрикам, то (наверное) можно говорить о состоятельности моделирования. Но при разных запусках они очень варьируются, поэтому стабильности нет => нет состоятельности моделирования. Визуализация позволит оценить не только факт возрастания или убывания расхождения, но и его абсолютную и относительную величину.

Код для визуализации полностью составлен с использованием ИИ (поэтому приводить его здесь не вижу смысла)

Визуализация подтвердила несостоятельность выборки - при увеличении ее размера (до 200) не наблюдается что “оценка будет стремиться к исходному значению параметра”.
При исключении из ГС нулевых значений, то есть оценка только покупателей исходный массив не population (5000 строк), а data (1820 строк) ситуация существенно не изменилась

```
data = ecom[ecom['purchase_amount'] > 0]['purchase_amount'].values
```

результат становится с одной стороны более интересным - убывающих отклонений при разном количестве запусков больше

```
Среднее: возрастающих = 1 убывающих = 3
Дисперсия: возрастающих = 1 убывающих = 3
Стандартное отклонение: возрастающих = 1 убывающих = 3
```

Но визуализация рассеивает эти иллюзии

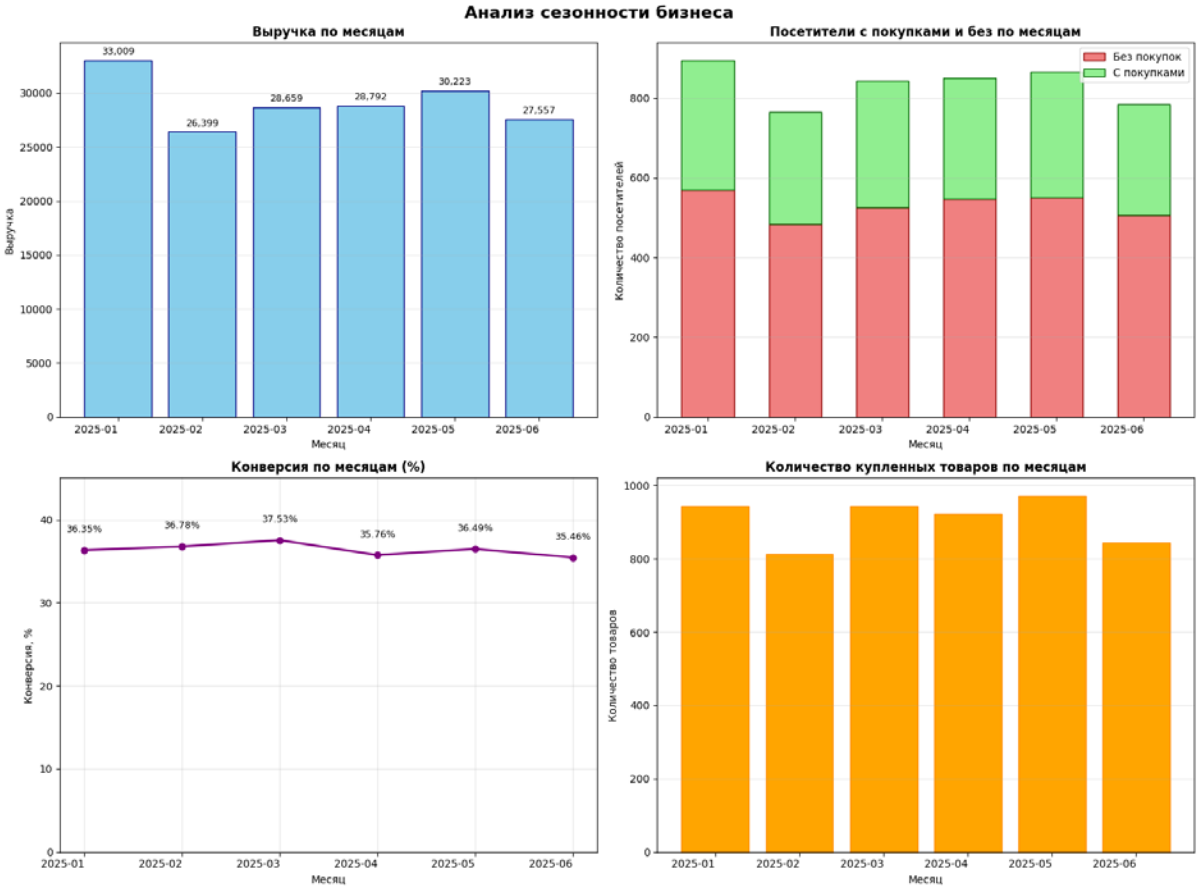
H. Бизнес-вывод

Прежде всего составим дизайн исследования после чего определим какими инструментами статистического анализа. Для бизнесовой части проекта создадим отдельный блокнот

1) определить наличие сезонности бизнеса (количество посетителей и покупателей по месяцам). применить метод ANOVA (для грубой оценки) различий. *(без детализации по апостероидным множественным сравнениям для выброк с разной дисперсией)*
   (попробуем сгенерировать код Python с использованием ИИ + проверка в SPSS
2) если есть сезонность (всплески по месяцам, то отдельно изучить самый плохой по продажам (количество покупателей, выручка) и самый хороший месяцы. Если сезонности нет, то далее рассматриваем весь массив данных без привязки к датам (ИИ+Python)
3) Рассчитать по каждой рекламной компании: средний чек (включая нулевые продажи), средний чек среди покупателей, конверсию, применить метод ANOVA (для грубой оценки). Ранжировать компании по эффективности (ИИ+Python)
4) Рассчитать по каждому источнику трафика: средний чек (включая нулевые продажи), средний чек среди покупателей, конверсию, применить метод ANOVA (для грубой оценки). Ранжировать компании по эффективности (ИИ+Python)
5) Оценить гендерные различия по критериям: конверсия, средний чек покупки (среди покупателей), время потраченное на сайте, число просмотренных страниц, количество купленных товаров. Оценить значимость различий на основе t-test (ИИ+Python)
6) Оценить влияние времени проведенного на сайте и глубины посещения сайта на сумму чека (линейная регрессия) (SPSS)
7) Рассчитать NPS (5 - промоутер, 4 - нейтрал, 1-3 - критик) по месяцам (Excel + SPSS)

Оценка наличия сезонности

Python

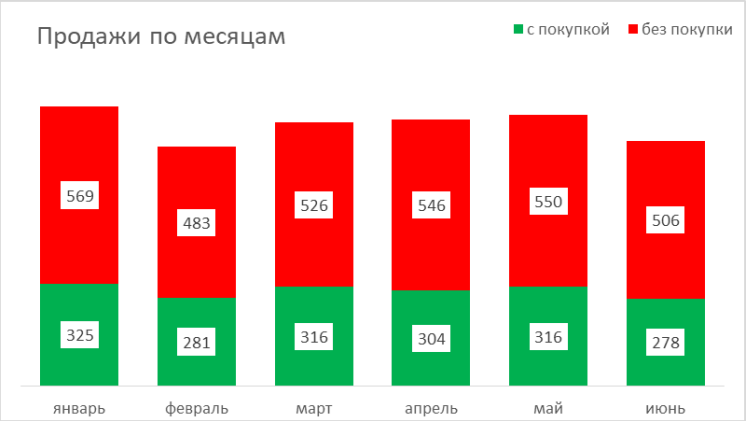


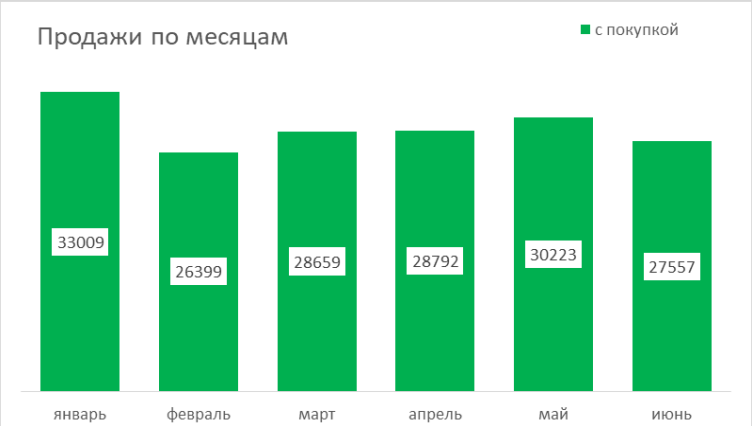
Можно констатировать отсутствие сезонности, что подтверждается как визуально, так и с помощью однофакторного дисперсионного анализа ANOVA

```
ANOVA тест на различия конверсии между месяцами:
  F-статистика: 0.1925
  p-значение: 0.9655
  Вывод: Нет статистически значимых различий между месяцами (p ≥ 0.05)
         Это может указывать на отсутствие выраженной сезонности.

Коэффициент вариации выручки по месяцам: 7.91%
  Низкий коэффициент вариации (<15%) - стабильный бизнес

======================================================================
ИТОГИ ПО СЕЗОННОСТИ:
Всего месяцев в данных: 6
Месяц с максимальной выручкой: 2025-01 (33,009)
Месяц с минимальной выручкой: 2025-02 (26,399)
Месяц с максимальной конверсией: 2025-03 (37.53%)
Средняя месячная конверсия: 36.40%
```

Для чистоты эксперимента сделал аналогичные выводы и манипуляции, привычными

ранее способами Excel + SPSS





Это понятнее, потому что понимаешь что делаешь с точностью до шага. Но дольше при том же результате. Вывод ИИ+Python работают очень хорошо. Код очень длинный, руками писать это долго и стоит все-равно проверять.

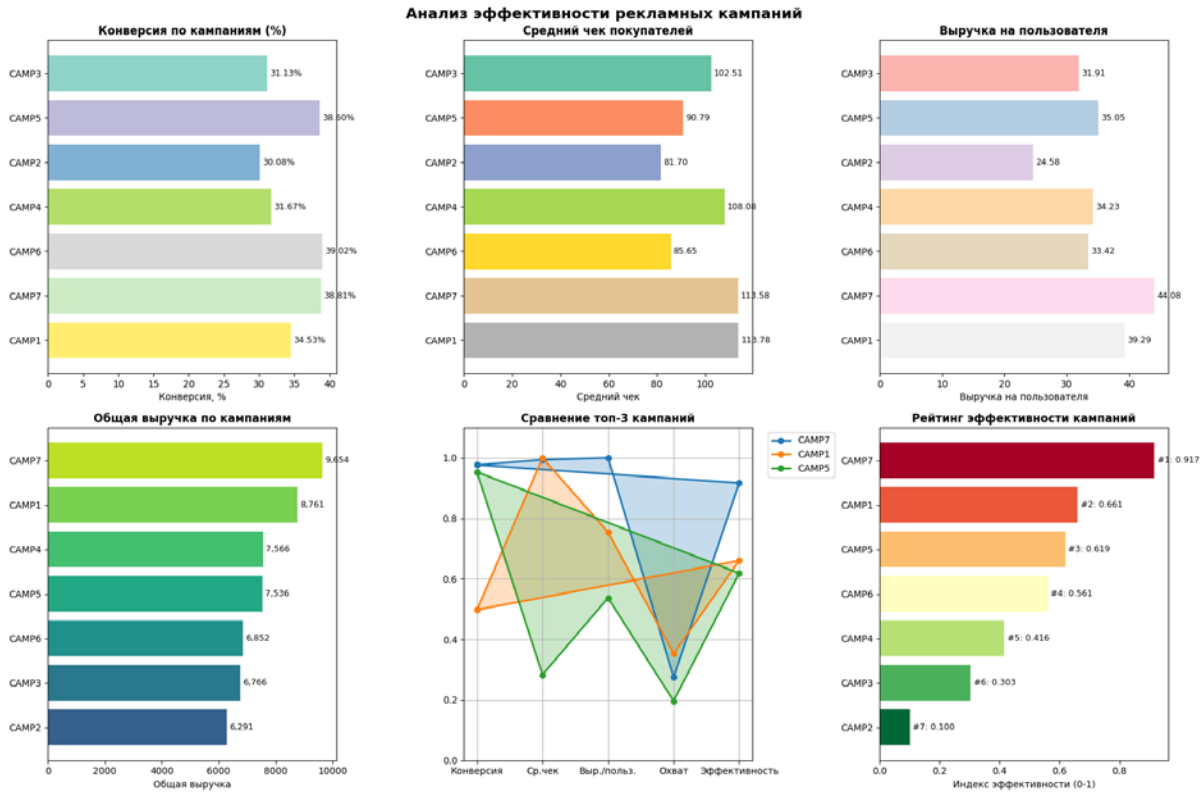
```
USE ALL.
COMPUTE filter_$=(purchase_amount>0).
VARIABLE LABELS filter_$ 'purchase_amount>0 (FILTER)'.
VALUE LABELS filter_$ 0 'Not Selected' 1 'Selected'.
FORMATS filter_$ (f1.0).
FILTER BY filter_$.
EXECUTE.

ONEWAY purchase_amount BY month
  /MISSING ANALYSIS.
```

Однофакторный дисперсионный анализ

сумма покупки

	Сумма квадратов	ст.св.	Средний квадрат	F	Знч.
Между группами	23418,245	5	4683,649	,310	,907
Внутри групп	27386414,1	1814	15097,251		
Итого	27409832,4	1819			

Оценка эффективности каждой рекламной кампании



Прошу прощения за “радугу” это продолжение эксперимента с ИИ пока вообще без участия в кодинге, но с проверкой расчетных величин. Расчеты все совпадают. ANOVA тест значимых различий не выявил.

Ранг	Кампания	Всего пользователей	Покупатели	Конверсия, %	Ср. чек покупателей	Общая выручка	Ср.чек посетителей	Индекс эффективности
1	CAMP7	219	85	38,81	113,58	9654,04	44,08	0,92
2	CAMP1	223	77	34,53	113,78	8761,27	39,29	0,66
3	CAMP5	215	83	38,6	90,79	7535,52	35,05	0,62
4	CAMP6	205	80	39,02	85,65	6851,85	33,42	0,56
5	CAMP4	221	70	31,67	108,08	7565,65	34,23	0,42
6	CAMP3	212	66	31,13	102,51	6765,53	31,91	0,3
7	CAMP2	256	77	30,08	81,7	6291,28	24,58	0,1

При расчете индекса эффективности используются 4 ключевых критерия с разными весами:

- Конверсия (вес: 0.4)
- Средний чек посетителя (вес: 0.3)
- Средний чек покупателей (вес: 0.2)
- Охват (Всего пользователей) (вес: 0.1)

Самая эффективная из рекламных кампаний (даже без учета значимости различий) **camp7**, худшая кампания **camp2**

Показатели **camp7**

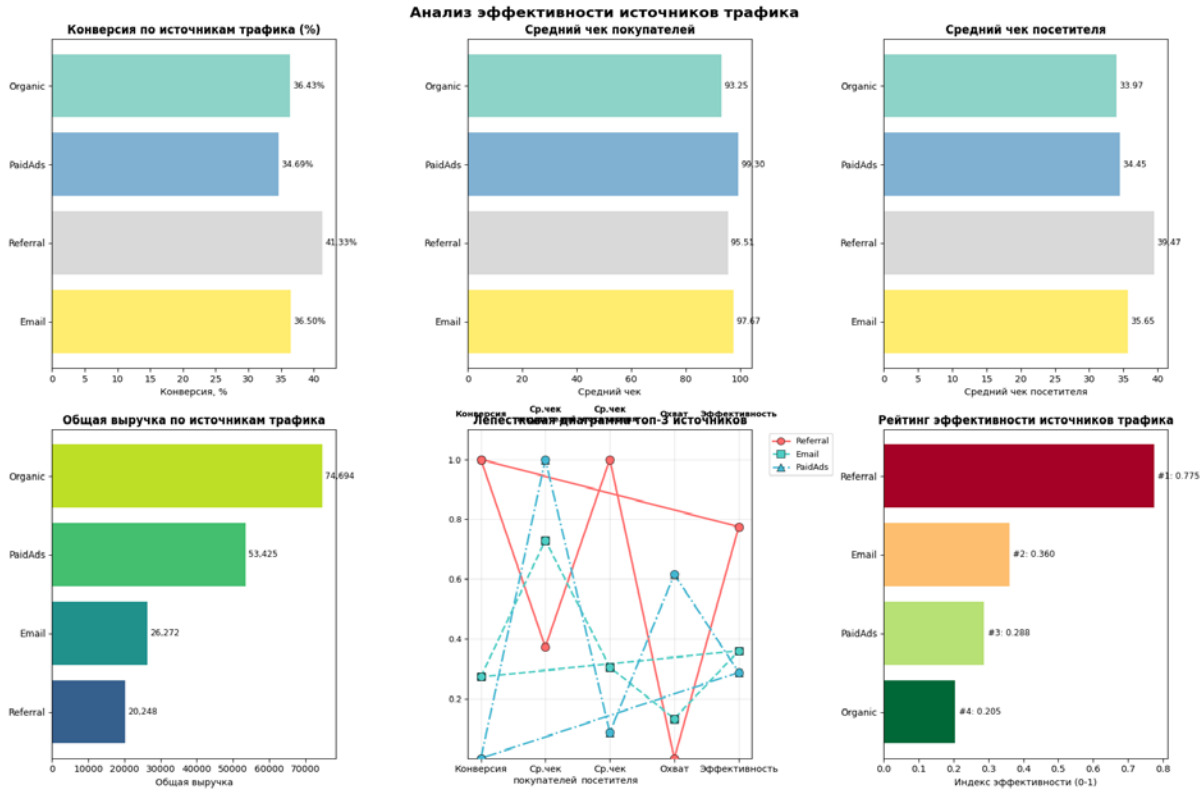
- Конверсия: 38.81%
- Средний чек покупателей: 113.58
- Выручка на пользователя: 44.08

Показатели **camp2**

- Конверсия: 30.08%
- Средний чек покупателей: 81.70

При этом **конверсия** среди посетителей **без рекламной кампании** на таком же уровне - **37.17%**. **Целесообразность проведения рекламных кампаний сомнительна**.

Оценка эффективности каждого трафика



Нет статистически значимых различий по каждой метрике (ANOVA) не выявлено

Ранг	Источник	Всего	Покупател	Конверси	Ср,чек	Ср,чек	Общая	Ср,чек	Индекс
1	Referral	513	212	41,33	39,47	95,51	20247,55	39,47	0,77
2	Email	737	269	36,50	35,65	97,67	26272,25	35,65	0,36
3	PaidAds	1551	538	34,69	34,45	99,30	53425,14	34,45	0,29
4	Organic	2199	801	36,43	33,97	93,25	74693,91	33,97	0,20

ЛУЧШИЙ ИСТОЧНИК ТРАФИКА: Referral

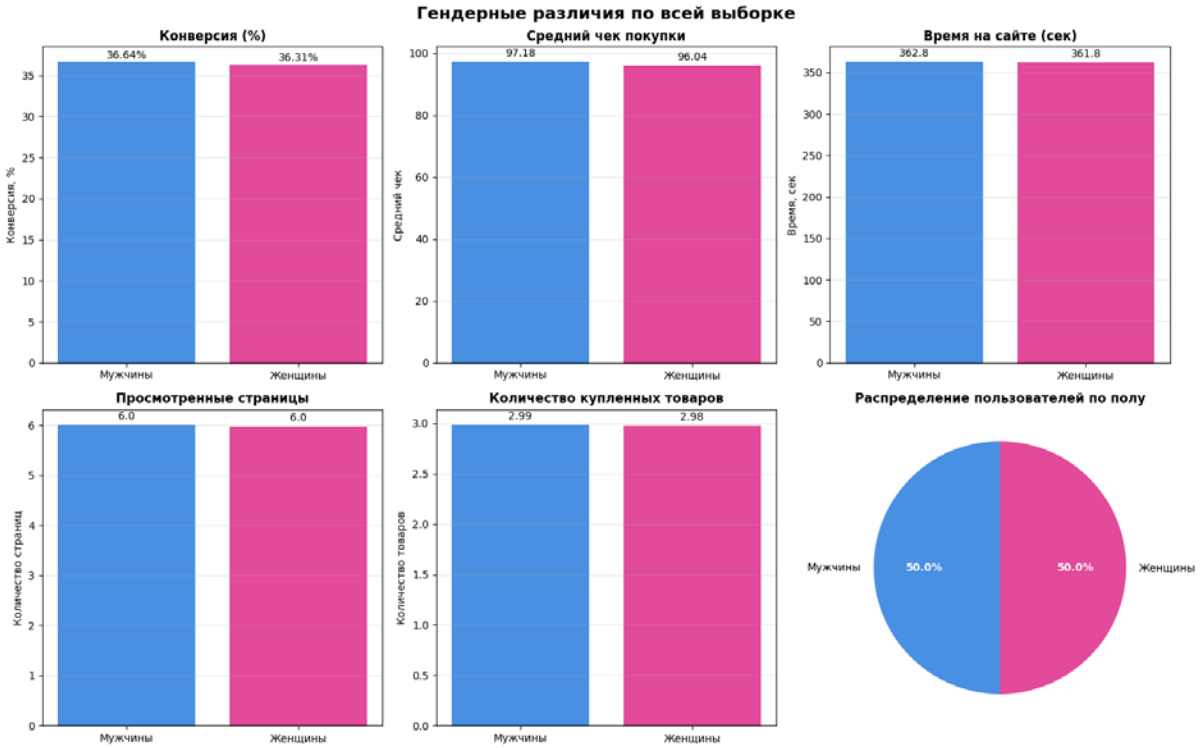
- Конверсия: 41.33%
- Средний чек покупателей: 95.51
- Средний чек посетителя: 39.47

ХУДШИЙ ИСТОЧНИК ТРАФИКА: Organic

- Конверсия: 36.43%
- Средний чек покупателей: 93.25

Гендерные различия

Не выявлены




```
Статистика по всей выборке:
                          Male       Female
count                2440.000000  2440.000000
conversion_rate        36.639344    36.311475
avg_purchase_amount    97.177718    96.042777
avg_time_on_site      362.798607   361.824385
avg_pages_visited       6.007787     5.969672
avg_num_items           2.987696     2.979684
buyers_count          894.000000   886.000000
```

Во-первых даже визуально не видны различия, во-вторых, проведенный t-test разумеется не подтвердил значимость несущественных различий:

```
Т-ТЕСТЫ (ПО ВСЕЙ ВЫБОРКЕ):
----------------------------------------------------
                metric  statistic  p_value significance
             Конверсия   0.237908 0.811952          н/з
           Средний чек   0.193937 0.846247          н/з
        Время на сайте   0.132940 0.894246          н/з
Просмотренные страницы   0.591210 0.554407          н/з
        Кол-во товаров   0.121342 0.903434          н/з

Примечание: *** - p < 0.001, ** - p < 0.01, * - p < 0.05, н/з - не
значимо
```

Кроме этого были посчитаны эти же метрики отдельно для топ 3 рекламных кампаний. Но там явные различия также не определены.

	CAMP7		CAMP1		CAMP5	
	Мужчины	Женщины	Мужчины	Женщины	Мужчины	Женщины
количество	110,00	104,00	109,00	113,00	99,00	106,00
конверсия	43,64	32,69	36,70	32,74	34,34	43,40
средний чек (покупатели)	112,27	109,19	111,03	116,76	97,68	89,22
время на сайте	380,74	365,40	409,75	359,58	380,06	402,11
средний чек (посетители)	6,15	6,00	6,08	6,20	5,96	6,26
срреднее количество купленных позиций	2,65	3,00	2,83	3,14	3,06	3,28

CAMP7: Мужчины конвертируются на 10.9% лучше
CAMP5: Женщины конвертируются на 9.1% лучше

Оценка зависимости суммы покупки от времени и глубины посещения сайта

Построена регрессионная модель (SPSS) с принудительным включением предикторов

```
FILTER OFF.
USE ALL.
EXECUTE.
LOGISTIC REGRESSION VARIABLES converted
  /METHOD=ENTER time_on_site_s pages_visited
  /CRITERIA=PIN(.05) POUT(.10) ITERATE(20) CUT(.5).
```

Сводка для модели

Модель	R	R-квадрат	Скорректированный R-квадрат	Стд. ошибка оценки
1	,218[a]	,048	,047	85,18235

a. Предикторы: (конст) число просмотренных страниц за визит., возраст пользователя., сколько секунд клиент провёл на сайте.

Для общей выборки достоверность модели очень низкая - 4,8%, при рассмотрении выборки только для покупателей purchase_amount>0 ситуация в лучшую сторону не изменилась. Для оценки факта совершения покупки (любая покупка) построена логистическая регрессия.

```
FILTER OFF.
USE ALL.
EXECUTE.
LOGISTIC REGRESSION VARIABLES converted
  /METHOD=ENTER time_on_site_s pages_visited
  /CRITERIA=PIN(.05) POUT(.10) ITERATE(20) CUT(.5).
```

Сводка для модели

Шаг	-2 Log Правдоподобие	R квадрат Кокса и Снелла	R квадрат Нэйджелкерка
1	5724,793[a]	,153	,210

a. Оценивание закончено на итерации номер 4, потому что оценки параметра изменились менее чем на ,001.

Достоверность модели выше чем в предыдущих моделях, но тоже очень низкая (15,3%), следовательно на факт принятия решения о покупке время проведенное на сайте и количество просмотренных страниц не влияет.

NPS по месяцам

Частота

		Месяц						Итого
		январь	февраль	март	апрель	май	июнь	
оценка удовлетворённости клиента.	2	2	0	0	2	1	0	5
	3	56	49	45	49	37	48	284
	4	180	168	185	181	191	159	1064
	5	87	64	86	72	87	71	467
Итого		325	281	316	304	316	278	1820
	NPS	8,9%	5,3%	13,0%	6,9%	15,5%	8,3%	9,8%

Наибольшая удовлетворенность в мае, возможно это коррелируется со скоростью доставки, но корреляции здесь нет

Среднее

Месяц	время доставки заказа (дни).
январь	8,07
февраль	7,71
март	7,80
апрель	8,15
май	7,66
июнь	7,94
Итого	7,89

Выводы и рекомендации

Похоже что этот учебный кейс содержит фейковые данные, которые были сгенерированы без учета каких-то мат моделей, для определенных групп пользователей. Единственные закономерности - сводятся к тому что их нет:

1. Рекламная кампания Camp7 - наиболее успешная и необходимо ее развивать, но без рекласных кампаний показатели те же самые
2. Рекламные кампании проводились только на трафик PaidAds
3. Разделения по соц.дему нет - целесообразность сегментации отсутствует
4. Связи между проведенным временем на сайте и покупками тоже отсутствует.